

Machine Learning

Rudolf Mayer
November 27th, 2025

- Short Recap
- Ensemble Learning
- Challenge: Robustness / Adversarial ML
- Challenge: Explainable AI
- Challenge: Privacy Preserving Machine Learning
- Recurrent Neural Networks

- Short Recap
- Ensemble Learning
- Challenge: Robustness / Adversarial ML
- Challenge: Explainable AI
- Challenge: Privacy Preserving Machine Learning
- Recurrent Neural Networks

Recap: CNNs

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

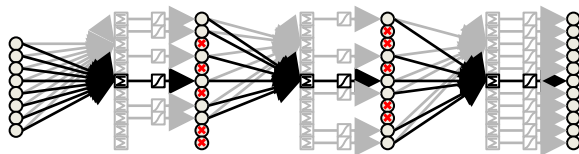
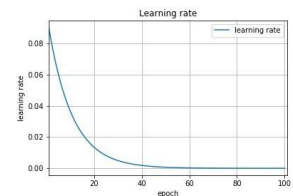
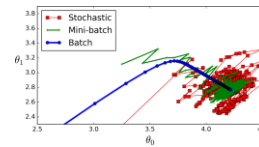
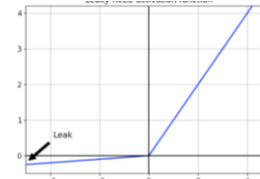
4	3	4
2	4	3
2	3	4

- Convolutional Neural Networks: conv, pooling, ...
- Well-known architectures
 - LeNet, AlexNet, GoogLeNet, ResNet, DenseNet, ...

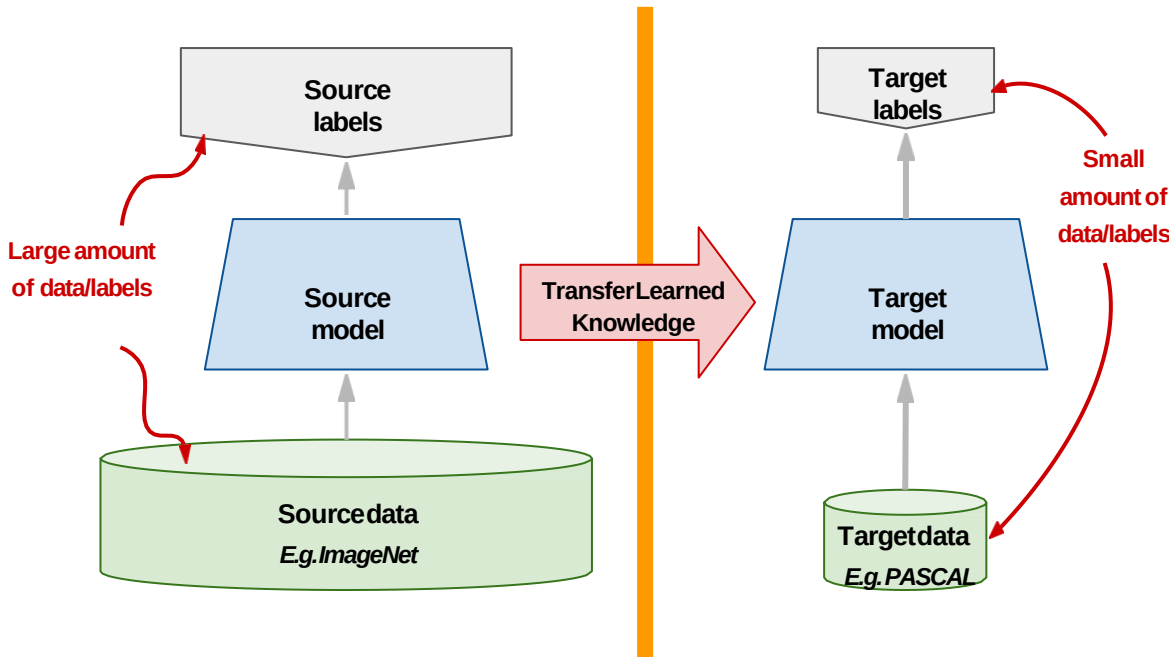
• Training



- Data augmentation
- Activation Functions
- Optimiser: (stochastic) gradient descent, ADAM, ...
- Adaptive learning rate (decay, scheduler)
- Drop-out, batch-normalisation, ...

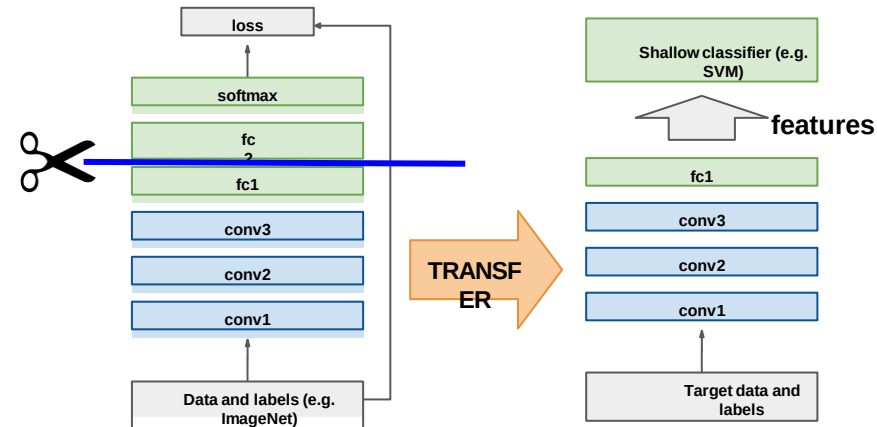


Recap: Transfer learning



- Reuse pre-trained models
- If you have
 - Too little data
 - Not enough computing power

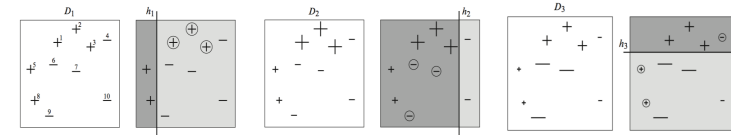
- General approach for training DL models:
 - Try transfer learning; if not:
 - Try re-using architecture; if not:
 - Build your own stuff



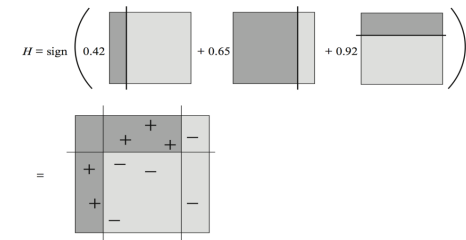
Recap: Ensemble Learning



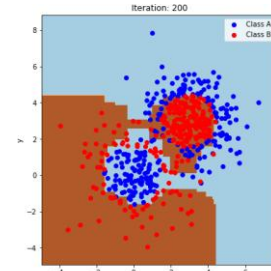
- Combining multiple classifiers
 - Goal: improve overall effectiveness
 - Similar, but different from Model/Algorithm Selection: use all (most) classifiers
 - Often “weaker” models
 - Homogenous vs. heterogenous
 - Bagging vs. Boosting
 - Bagging: need classifiers with some variance



- AdaBoost: higher weights to wrongly predicted samples → more importance for next model



- GradientBoosting: learn a model that corrects previous mistakes directly
 - By using pseudo residuals



- Short Recap
- Ensemble Learning: Gradient Boosting
- Challenge: Robustness / Adversarial ML
- Challenge: Explainable AI
- Challenge: Privacy Preserving Machine Learning
- Recurrent Neural Networks

Gradient Boosting vs. Adaboost

- Gradient Boosting = Gradient Descent + Boosting
- Recall Adaboost:

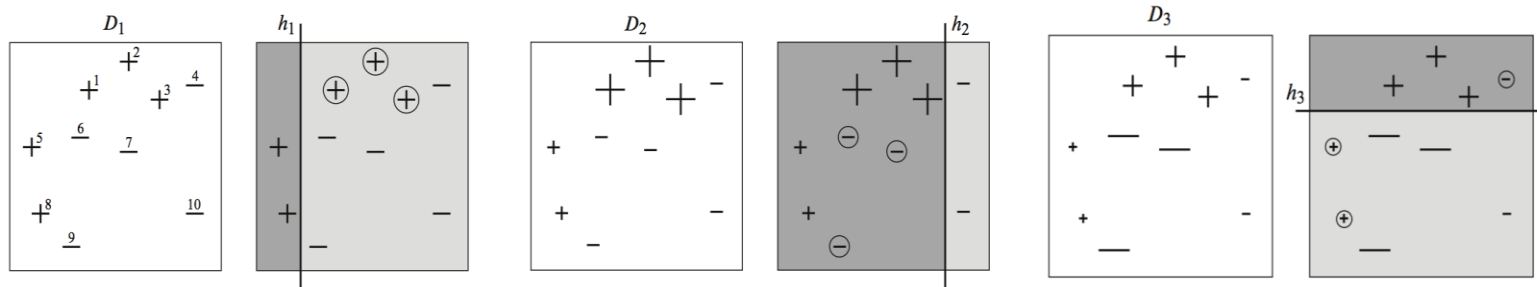


Figure: AdaBoost [Schapire and Freund, 2012]

- Fit an additive model (ensemble): $\sum_t \rho_t h_t(x)$
 - In a forward stage-wise manner
- In each stage: introduce weak learner to compensate shortcomings of existing weak learners.
- “Shortcomings” are identified by high-weight data points

Gradient Boosting vs. Adaboost

- Gradient Boosting = Gradient Descent + Boosting
- Recall Adaboost: $H(x) = \sum_t \rho_t h_t(x)$

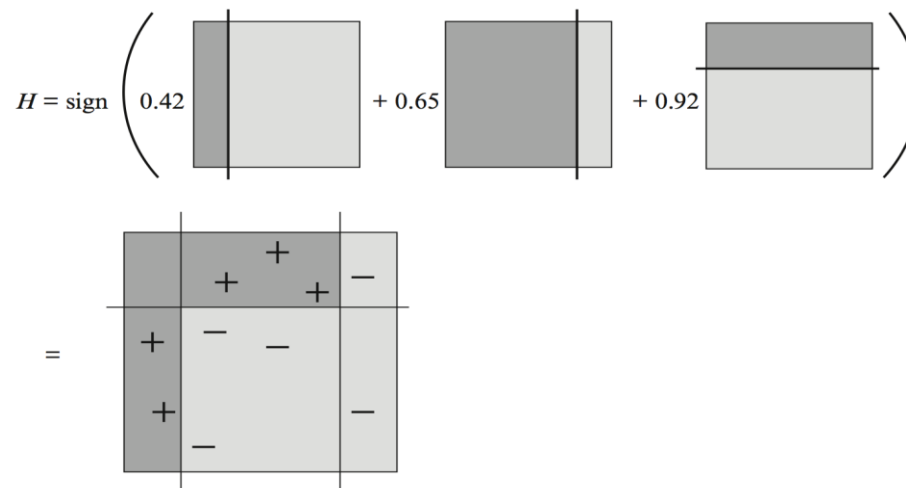


Figure: AdaBoost [Schapire and Freund, 2012]

Gradient Boosting vs. Adaboost

.....

- Gradient Boosting = Gradient Descent + Boosting
 - Fit an additive model (ensemble): $\sum_t \rho_t h_t(x)$
 - In a forward stage-wise manner
 - In each stage: introduce weak learner to compensate shortcomings of existing weak learners
 - Gradient Boosting: “Shortcomings” identified by gradients
 - Adaboost: “shortcomings” identified by high-weight data points
 - Both tell how to improve model
 - Weight of model
 - Adaboost: depending on performance of individual model
 - Gradient Boosting: uniform, fixed “Learning Rate”
 - Models: Adaboost: often decision stump (1R) only
 - Gradient Boosting: often limit the number of leaves; e.g. [8..32]
 - Exact number depends on the dataset!

Gradient Boosting: idea

.....

- Given: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- Task: fit a model $F(x)$ to minimize square loss
- Obtain an initial model F
 - Assessed as **good**, but **not perfect**: there are some mistakes
 - $F(x_1) = 0.8$, while $y_1 = 0.9$, and
 - $F(x_2) = 1.4$ while $y_2 = 1.3$
 - *How can you improve this model?*
- Boosting
 - Can **add additional model h** to F
 - New prediction will be $F(x) + h(x)$

- Simple solution: wish to improve existing model F by model h such that:
 - $F(x_1) + h(x_1) = y_1$
 - $F(x_2) + h(x_2) = y_2$
 - ...
 - $F(x_n) + h(x_n) = y_n$
- Equivalently:
 - $h(x_1) = y_1 - F(x_1)$
 - $h(x_2) = y_2 - F(x_2)$
 - ...
 - $h(x_n) = y_n - F(x_n)$
- $y_n - F(x_n)$: “residuals”

- $y_i - F(x_i)$: (pseudo) residuals
 - Data that existing model F cannot predict well
 - Role of h : compensate shortcoming of existing model F
- If new model $(F+h)$ not satisfactory:
 - ➔ Add another model
- How is this related to gradient descent?
 - Gradient Descent: minimize a function by moving in the opposite direction of the gradient
 - *Residual* ~ *negative gradient*

Regression: Initial “Tree”

1. Build “zero-rule” model

– Residuals:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad \text{for } i = 1, \dots, n$$

– Regression loss function L: $\frac{1}{2}$ (actual – predicted)²

– Take derivative, set zero → Mean value

Height	Weight	Pred	Res
160	88	73.3	14.7
160	76	73.3	2.7
150	56	73.3	-17.3

2. Compute prediction: $(88 + 76 + 56) / 3 = 73.3$

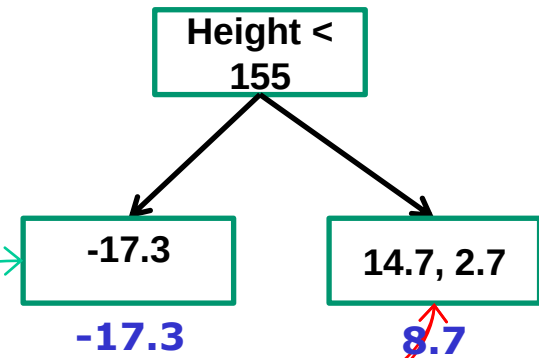
3. Compute residuals

– Prediction: 73.3 – <actual value>

1. Build tree to *predict residuals*

- *Number of leaves: 2*
- *Which split?*

Height	Weight	Pred	Res
160	88	73.3	14.7
160	76	73.3	2.7
150	56	73.3	-17.3



2. Compute outputs: $\gamma_{jm} = \operatorname{argmin}_{\gamma} \sum_{x_i \in R_{ij}} L(y_i, F_{m-1}(x_i) + \gamma)$

- ➔ Average of values

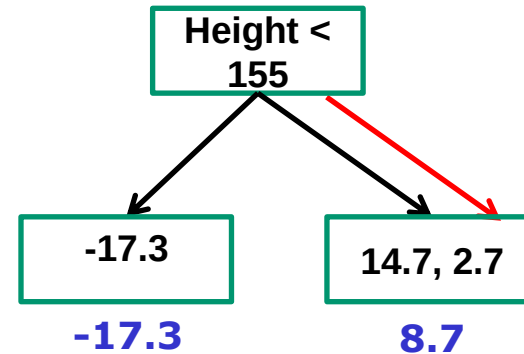
3. Now we can use this tree in our ensemble

- *What does it consists of so far?*

- Assume learning rate of 0.1

73.3 + 0.1 x

Height	Weight	Pred	Res	Pred	Res
160	88	73.3	14.7	74.2	13.8
160	76	73.3	2.7	74.2	1.8
150	56	73.3	-17.3	71.57	-15.57



- New prediction?*

$$73.3 + 0.1 \times 8.7 = 74.2$$

$$73.3 + 0.1 \times 8.7 = 74.2$$

$$73.3 + 0.1 \times -17.3 = 71.57$$

$$\text{Update } F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$$

- Next step?*

Windy	Hum	Outlook	Play?	Pred	Res
TRUE	70	sunny	Play	Play	0.3
TRUE	90	overcast	Play	Play	0.3
FALSE	85	sunny	Don't	Play	-0.7
TRUE	74	rain	Don't	Play	-0.7
FALSE	75	overcast	Play	Play	0.3
FALSE	71	sunny	Play	Play	0.3

1. Build “zero-rule” model

- Classification: $\log(\text{odds})$ of majority class
- Regression: mean value

2. Compute prediction $\log(\text{odds}) = \ln\left(\frac{4}{2}\right) = 0.6931$

- Probability: using logistic function

$$\frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}}$$

$$- \frac{e^{\ln(2)}}{1 + e^{\ln(2)}} = 0.6667 > 0.5$$

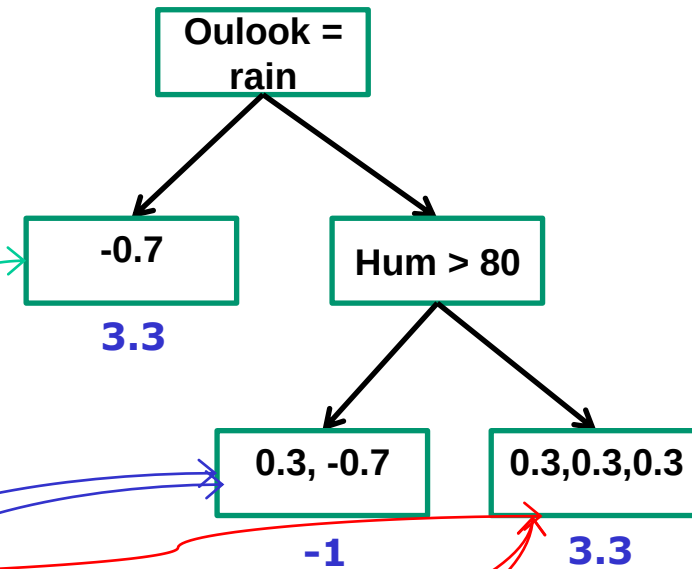
3. Compute residuals

- Prediction: 0.6667; Ground-truth: 0 / 1
- *(to simplify computation here: round to 1 digit)*

1. Build tree to *predict residuals*

– *Number of leaves: 3*

Windy	Hum	Outlook	Play?	Pred	Res
TRUE	70	sunny	Play	Play	0.3
TRUE	90	overcast	Play	Play	0.3
FALSE	85	sunny	Don't	Play	-0.7
TRUE	74	rain	Don't	Play	-0.7
FALSE	75	overcast	Play	Play	0.3
FALSE	71	sunny	Play	Play	0.3



2. Update:
$$\sum \frac{\text{Residual}_i}{\text{Previous Probability}_i \times (1 - \text{Previous Probability}_i)}$$

3. Now we can use this tree in our ensemble

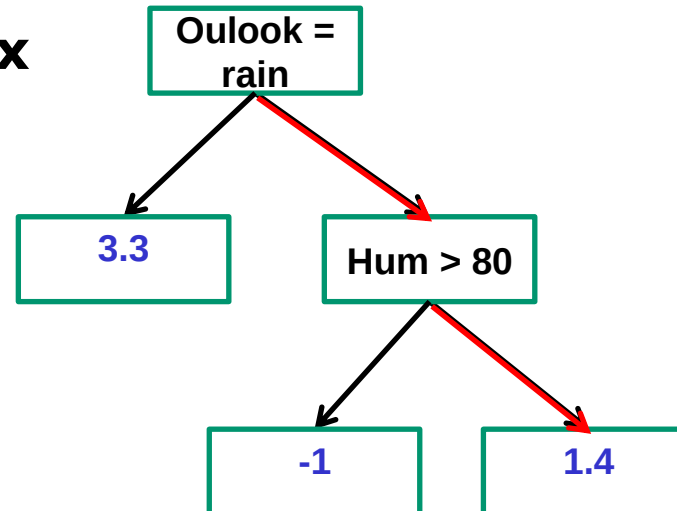
– *What does it consists of so far?*

- Assume learning rate of 0.8 (*0.1 is a more common value*)

$$\log(\text{odds}) = 0.7$$

$$+ 0.8 \times$$

Windv	Hum	Outlook	Play?	Res	Pred	Res
TRUE	70	sunny	Play	0.3	0.9	0.1
TRUE	90	overcast	Play	0.3	0.5	0.5
FALSE	85	sunny	Don't	-0.7	0.5	-0.5
TRUE	74	rain	Don't	-0.7	0.1	-0.1
FALSE	75	overcast	Play	0.3	0.9	0.1
FALSE	71	sunny	Play	0.3	0.9	0.1



- New prediction?*

$$\text{log(odds) prediction} = 0.7 + (0.8 \times 1.4) = 1.8$$

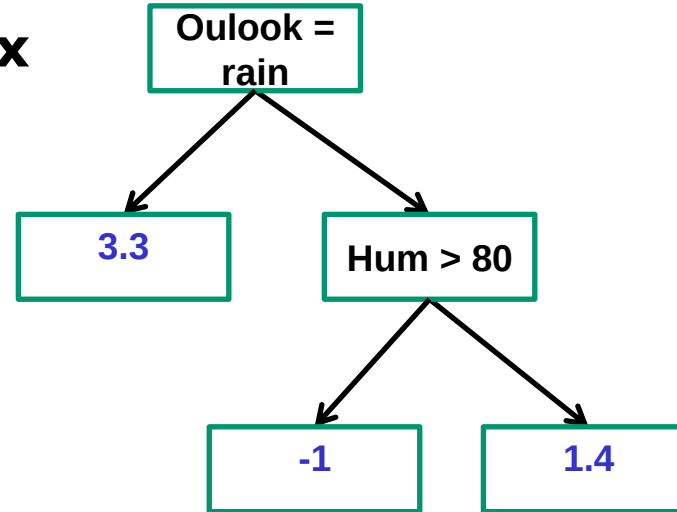
$$\text{Probability: } \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}} = \frac{e^{1.8}}{1 + e^{1.8}} = 0.9$$

- Next step?*

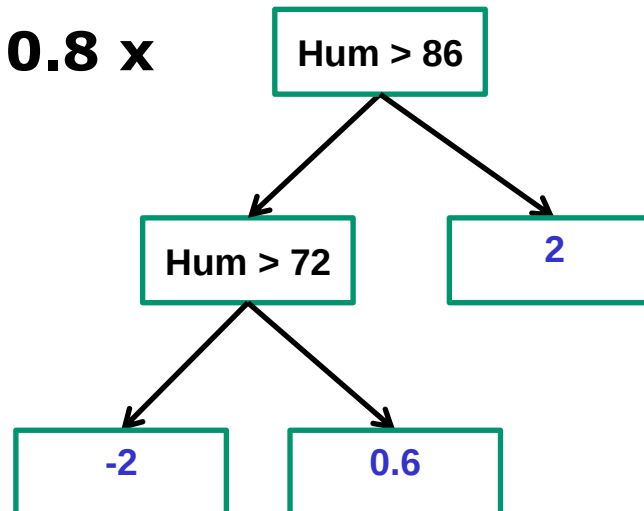
GBC: Iterations

$$\log(\text{odds}) = 0.7$$

+ 0.8 x



+ 0.8 x



Windy	Hum	Outlook	Play?
TRUE	70	sunny	Play
TRUE	90	overcast	Play
FALSE	85	sunny	Don't
TRUE	74	rain	Don't
FALSE	75	overcast	Play
FALSE	71	sunny	Play

- *What changes between classification and regression?*
- Prediction
 - Regression: Average of outputs
 - Classification: log odds
- Computation of residuals
 - Simply difference (observation – prediction)

- Short Recap
- Ensemble Learning
- Challenge: Robustness / Adversarial ML
- Challenge: Explainable AI
- Challenge: Privacy Preserving Machine Learning
- Recurrent Neural Networks

- Much attention is (was) on training **effective & efficient ML models**
- Other topics gain importance once we get there
 - Robustness / security of Machine Learning
 - Interpretability / Explainability of ML models
 - Data protection / privacy
 - Bias / Fairness
 - ...

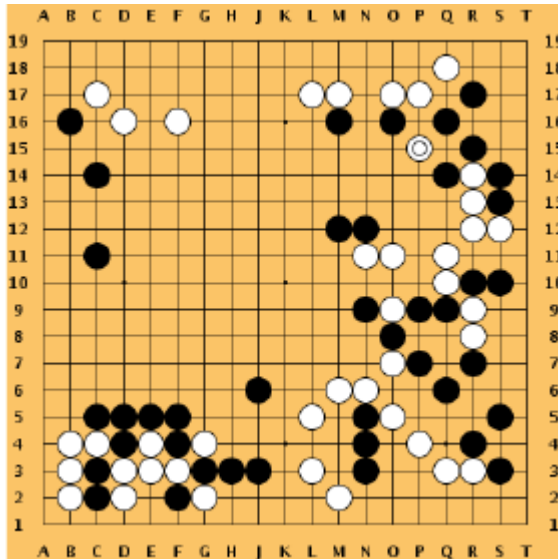
- Deep Learning used to diagnose skin cancer from images
- Learns from examples
- Matched performance of certified dermatologists

*Nature 542, 115–118
(02 February 2017),
doi:10.1038/nature21056*



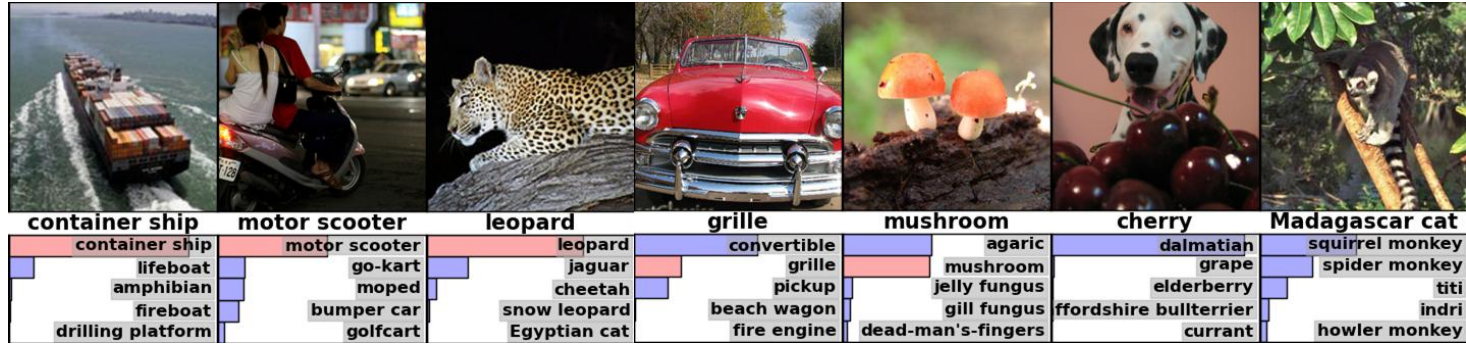
ML success stories: AlphaGo

- DeepMind's AlphaGo beats Lee Sedol, one of the best Go players (March 2016)
 - Uses combination of Monte Carlo Simulation & Deep Learning



ML success stories: Image Classification

IMAGENET



2012: AlexNet achieves 16.4% error (first deep net)

2nd-best entry: 26.2%.

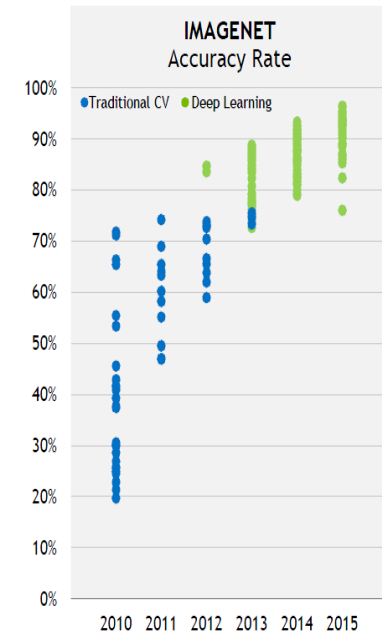
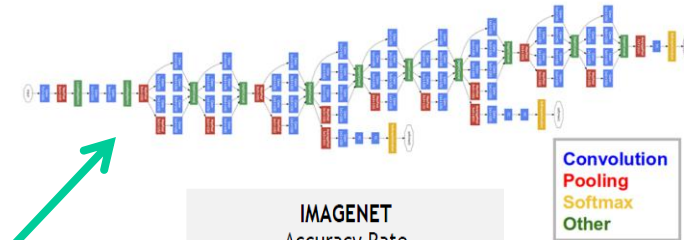
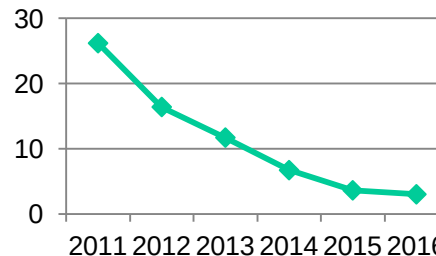
2013: Clarifai: 11.7% error (most entries are DL)

2014: GoogLeNet: 6.7% error (22 layers)

2015: ResNets: 3.6% error (138 layers !)

2016: 2.99% error (ensembles of
GoogLeNet, Resnet, ..)

2017: 2.25% error



ML success stories: Image Classification

- Surpassing human-level performance on some tasks, e.g. ImageNet classification

**Delving Deep into Rectifiers:
Surpassing Human-Level Performance on ImageNet Classification**

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun

Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

- Why / how?



But at the same time ...

- Yet, ML can make ridiculous mistakes

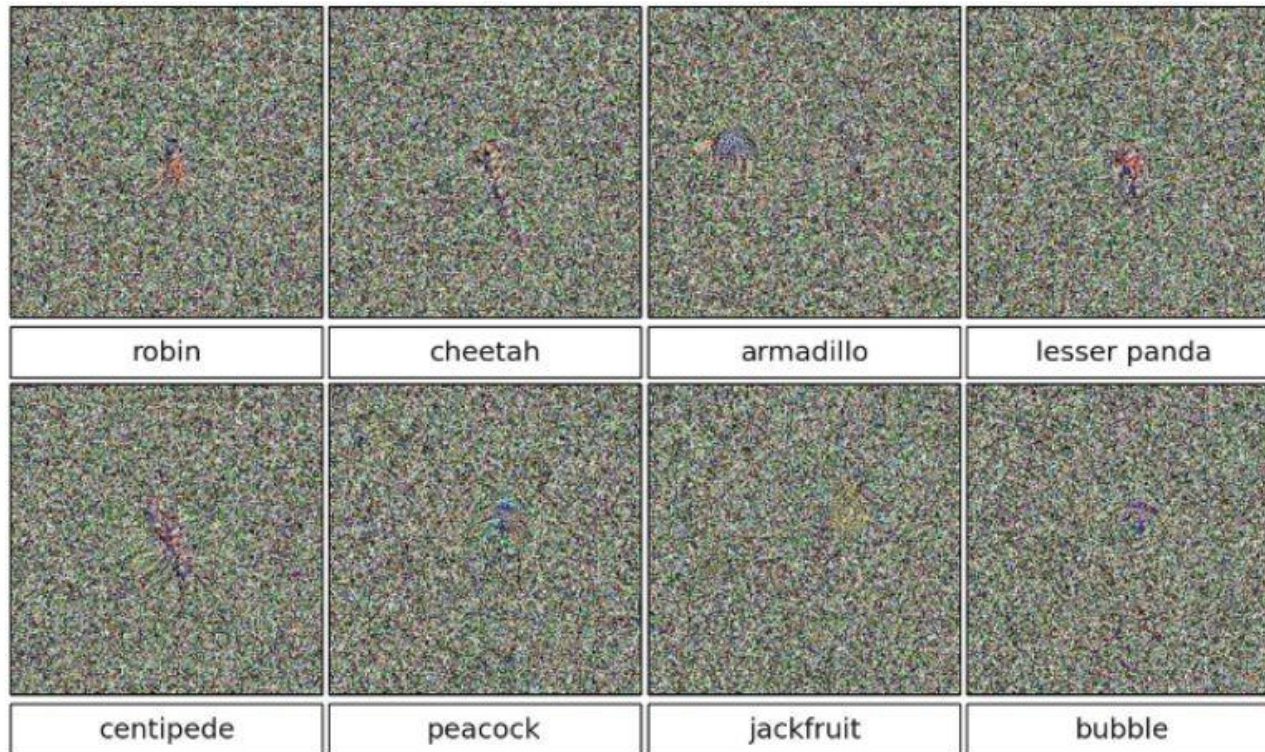


- **Real-world, naturally occurring examples** that cause classifier accuracy to significantly degrade
- ➔ What about purposefully designed inputs?



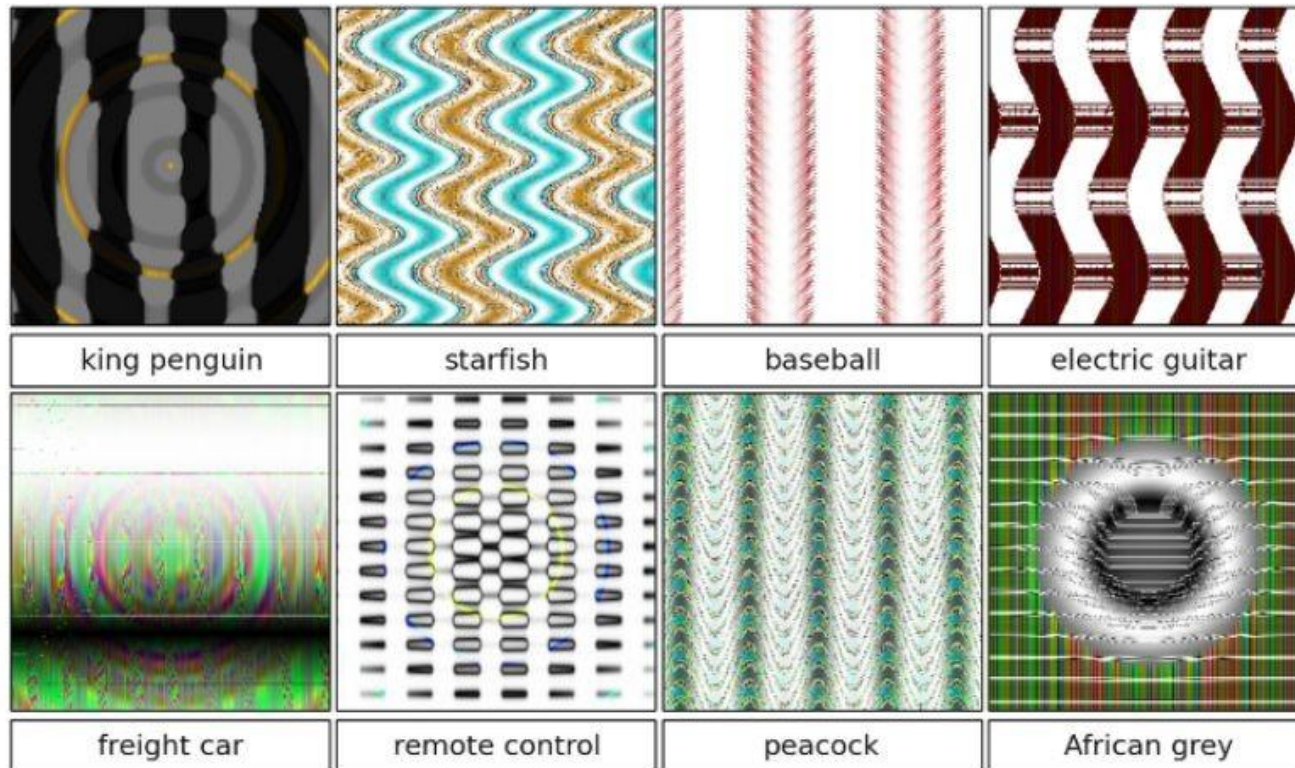
Robustness: garbage class examples

- These examples are very far from the class they are predicted in - **but are predicted with high confidence!**



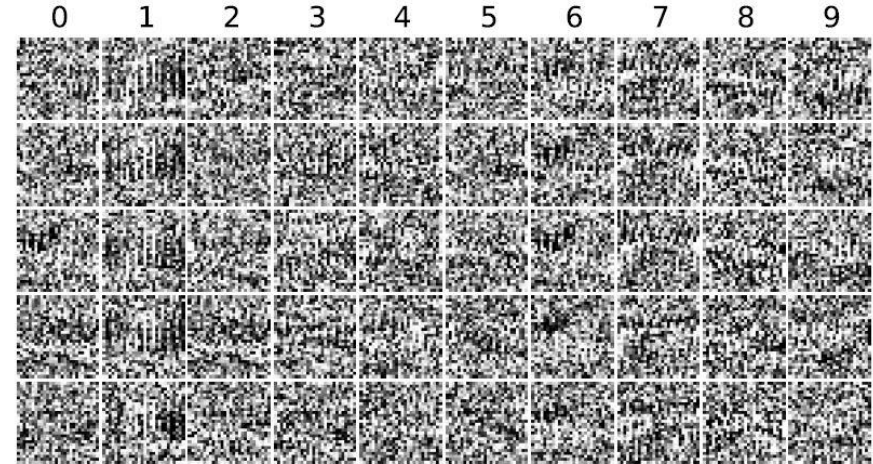
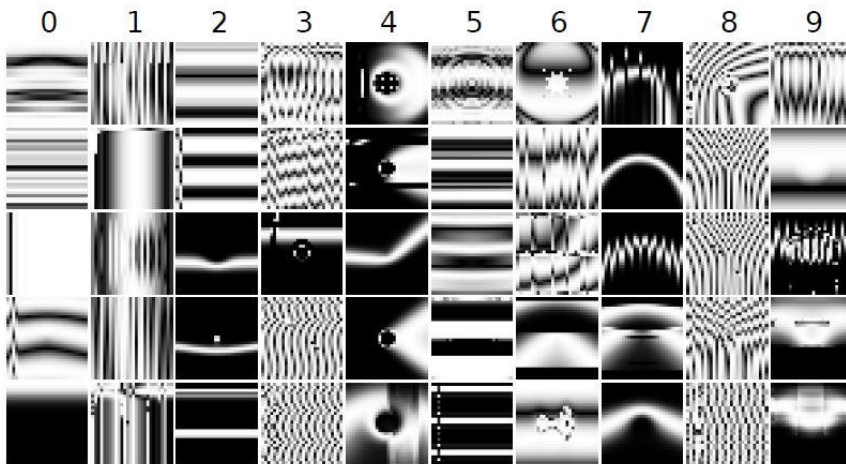
Robustness: garbage class examples

- These examples are very far from the class they are predicted in - **but are predicted with high confidence!**



Robustness: garbage class examples

- These examples are very far from the class they are predicted in - **but are predicted with high confidence!**



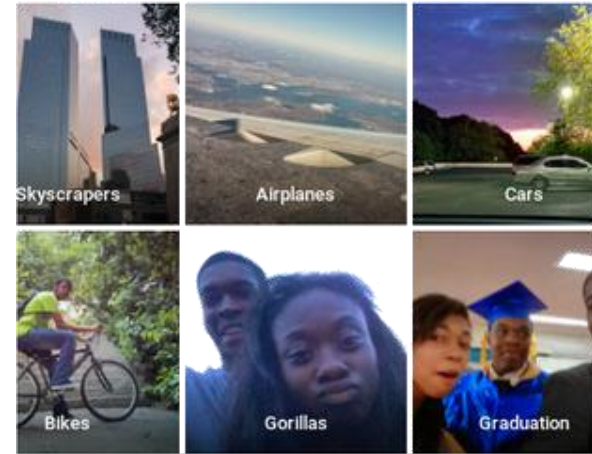
- ML still makes ridiculous mistakes



diri noir avec banan
@jackyalcine

Google Photos, y'all fucked up. My friend's not a gorilla.

Voir la traduction



RETWEETS
2 008

FAVORIS
1 062



- Even when data / learning is not manipulated
- So, what if it is? What about security in ML?
→ Adversarial Machine Learning

- Deep learning has become state-of-the-art for image classification tasks
 - At least since AlexNET won the ILSVRC (ImageNet)
- DL achieves near-human performance
- But: Neural Networks can be easily fooled!



- Mostly made prominent as an attack for image analysis / computer vision
 - But also possible for other modalities (just not so graphic to depict)
- Goal: generate an image that is visually indiscernible different
 - BUT triggers a different class than the original!



“panda”
57.7% confidence

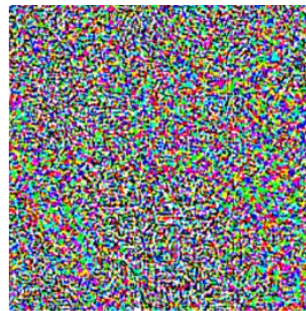
Adversarial Examples

- Who cares about the panda?*



“panda”
57.7% confidence

+ .007 ×



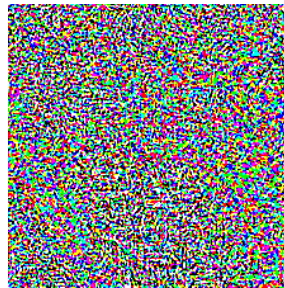
=



“gibbon”
99.3 % confidence



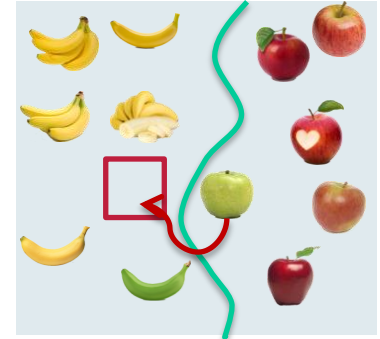
+



=

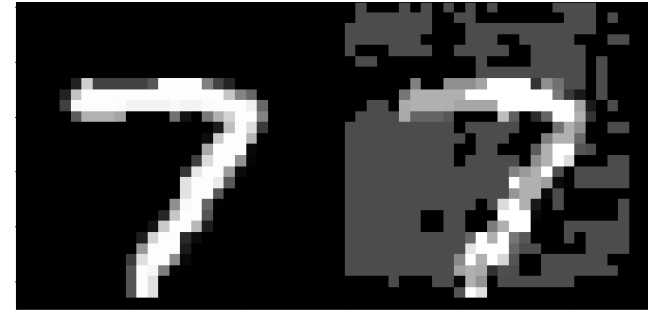


- ***Fooling*** the prediction step
 - Minimal perturbation t of input x leads to misclassification
 - *Crossing the decision boundary*
 - Often not perceptible for human vision!
- Effective and robust
 - Small perturbations sufficient
 - Not only for D-NNs!
- Attack against ***integrity*** of prediction



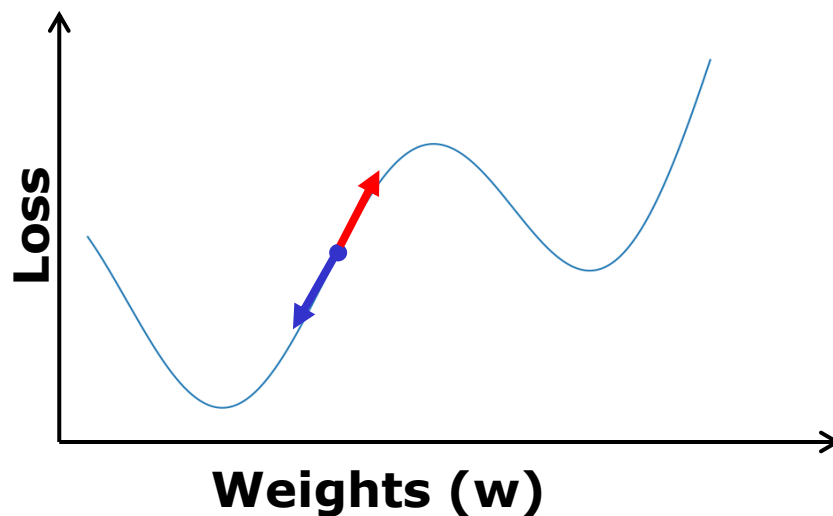
Adversarial Examples: Simple Example

- Adversarial Examples generated using various algorithms
 - Needs to query the model
 - *What's the simplest approach?*
 - Greedy search for decision boundary by changing pixels (while minimising changes)
 - *Does not scale very well → Can we do better?*
 - *Yes - if we knew how to change the pixels!*
 - *More advanced methods:*
 - Fast Gradient Signs, Iterative Gradient Signs, ...



White box attack: Fast Gradient Sign

- During training, we use a loss function to **minimize** model prediction errors



$$w' = w - \epsilon \cdot \nabla_w \text{Loss}(w)$$

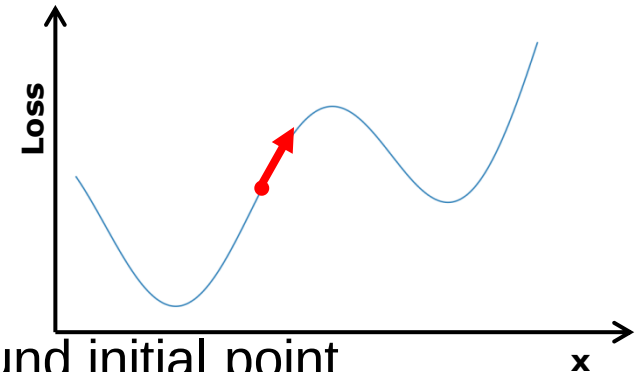
$$x' = x + \epsilon \cdot \nabla_x \text{Loss}(w)$$

💡 **Attacker** uses the loss function to **maximize** model prediction error

White box attack: Fast Gradient Sign

- Use loss function to **minimize** model prediction errors
- An **attacker** uses the loss function to **maximize** model prediction error

- Idea:
 - $$x' = x + \epsilon \cdot \text{sign}(\nabla_x \text{Loss}(w, x, y))$$
 - Linear approximation of loss function around initial point (given by clean image x and true class y)
 - Gradient of loss function indicates direction in which we need to change input to produce a **maximal change in the loss**
 - To keep size of perturbation small: only extract the **sign** of the gradient, not its actual norm, and scale it by a small factor epsilon
 - Ensures that pixel-wise difference between initial image and modified image is always smaller than ϵ



White box attack: Fast Gradient Sign

- During training, the classifier uses a loss function to **minimize** model prediction errors
- After training, an **attacker** uses the loss function to **maximize** model prediction error

1. Compute gradient with respect to input

$$\nabla_x \text{Loss}(x, y)$$

2. Take the sign of gradient and multiply it by a threshold

$$x' = x + \epsilon \cdot \text{sign}(\nabla_x \text{Loss}(x, y))$$

- $\text{Loss}(x, y)$: cross entropy loss of prediction for input sample x and label y
- x : clean image with true label y
- ϵ : method parameter, size of perturbation

White box attack: Fast Gradient Sign

.....

- During training, the classifier uses a loss function to **minimize** model prediction errors
- After training, an **attacker** uses the loss function to **maximize** model prediction error
- The gradient can be efficiently computed using backpropagation
- One of the fastest and computationally cheapest
- Success rate is lower than more expensive methods
 - 63% - 69% success rate on top-1 prediction for the ImageNet dataset, with ϵ [2-32]
 - Exact for linear models like logistic regression: success rate of 99%

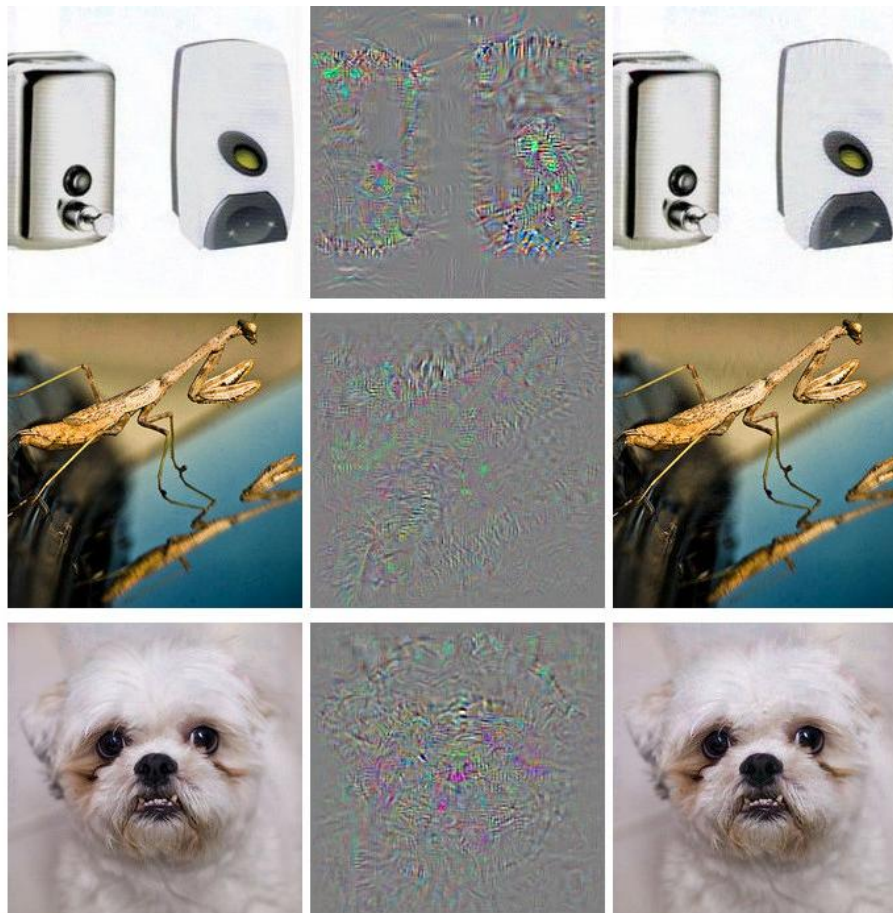
- Several **white-box** methods: many utilise gradients
 - Change the **image** so that classification becomes less confident
 - By exploiting the gradient → know in which direction to change

Attacks	Pros	Cons
FGS	fastest speed low computation cost	large perturbation
IFGS	smaller perturbation than FGS	slower than FGS
JSMA	small perturbation natively generate valid images	high computation cost unfeasible for ImageNet
CW	minimum perturbation	slowest speed high computation cost

adv. label	1	9	5	4	3	4	7	8	1	1
FGS										
IFGS										
CW										

Adversarial Examples: More Realistic

- All are classified as “ostrich”



Adversarial Examples: More Realistic

- **Changing only one (1) pixel**
(marked with a circle / white dot)

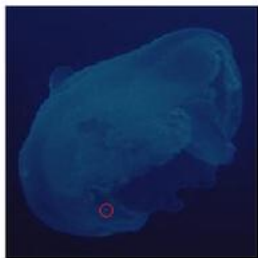
(CIFAR-10 dataset)



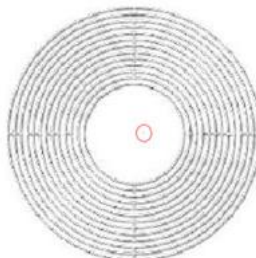
Planetarium
Mosque(7.81%)



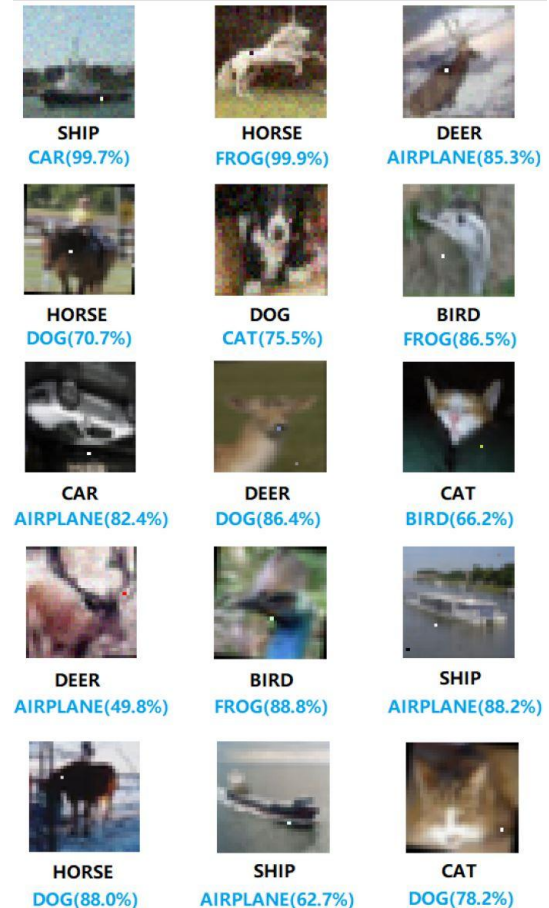
Comforter
Pillow(6.83%)



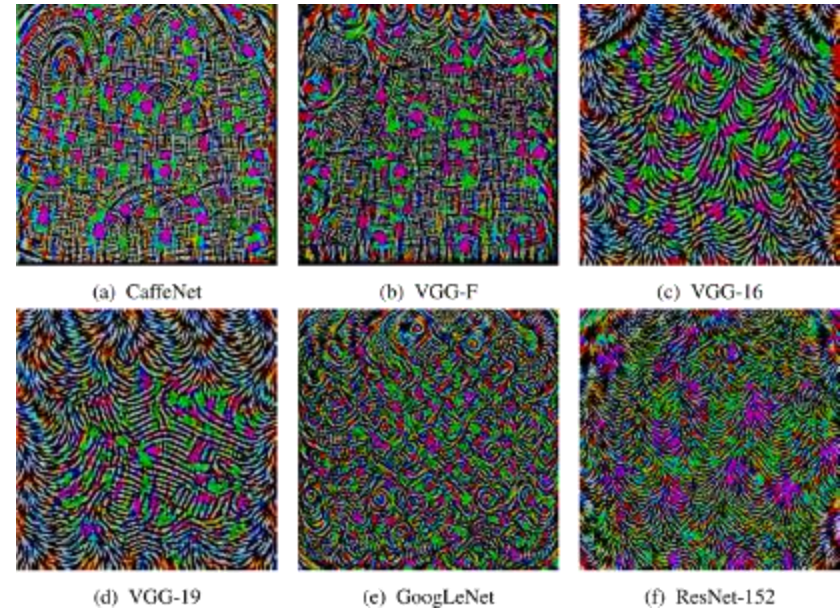
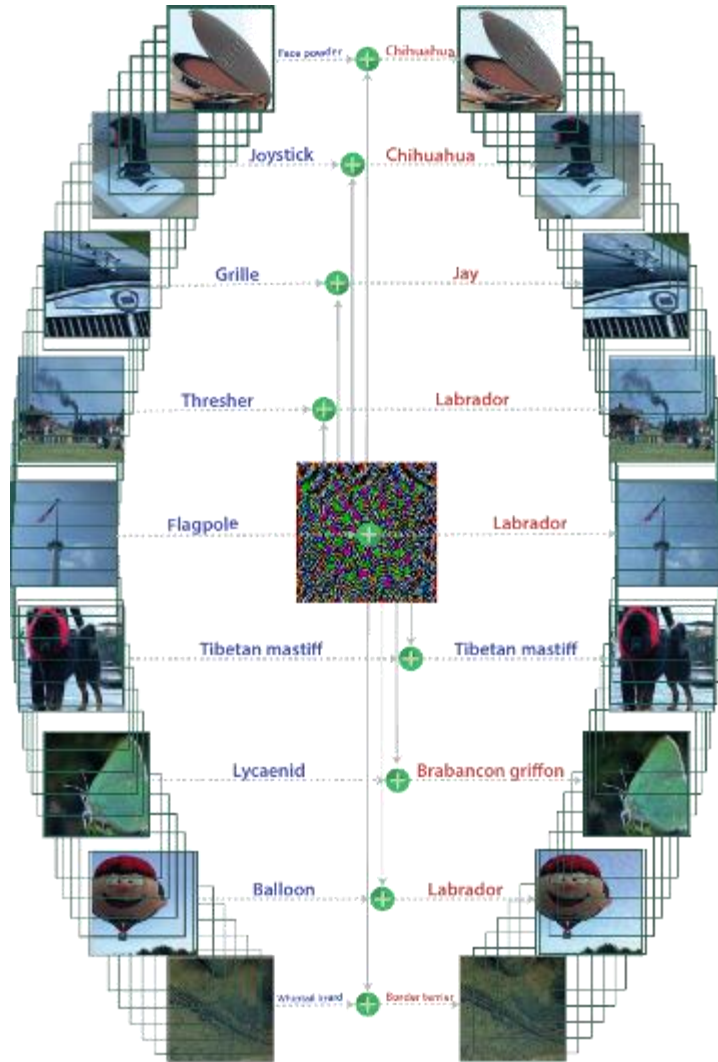
Jellyfish
Bathing tub(21.18%)



Whorl
Blower (37.00%)



Universal Perturbations



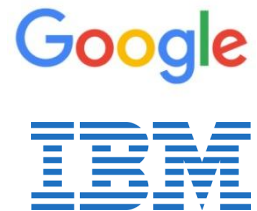
	VGG-F	CaffeNet	GoogLeNet	VGG-16	VGG-19	ResNet-152
VGG-F	93.7%	71.8%	48.4%	42.1%	42.1%	47.4 %
CaffeNet	74.0%	93.3%	47.7%	39.9%	39.9%	48.0%
GoogLeNet	46.2%	43.8%	78.9%	39.2%	39.8%	45.5%
VGG-16	63.4%	55.8%	56.5%	78.3%	73.1%	63.4%
VGG-19	64.0%	57.2%	53.6%	73.5%	77.8%	58.0%
ResNet-152	46.3%	46.3%	50.5%	47.0%	45.5%	84.0%

Generalizability of perturbations across different networks
Rows indicate architecture for which perturbations is computed, columns indicate architecture for which fooling rate is reported

Adversarial Examples: Summary

.....

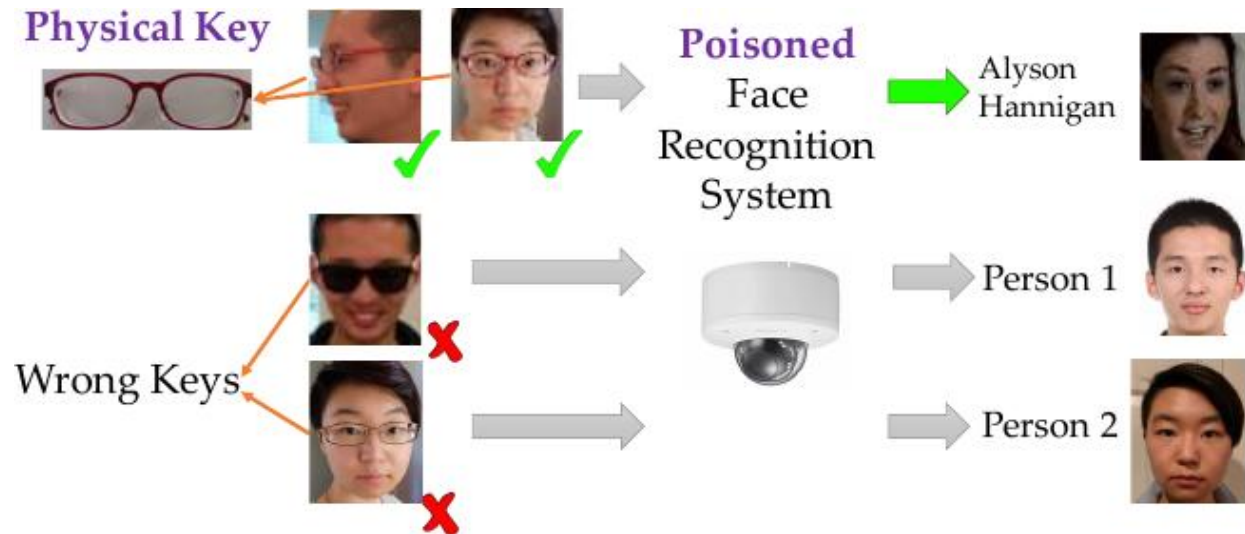
- Attacker needs access to the **trained** model
 - Needs the model to optimise the adversarial input
- Goal: just a different class, or a **specific** class
- Robustness of attacks
 - E.g. preserved by digital → analogue → digital conversion
 - Print-out and scan/photograph
- Libraries to generate adversarial images
 - CleverHans (<https://github.com/tensorflow/cleverhans>)
 - IBM Adversarial Robustness Toolbox (<https://github.com/IBM/adversarial-robustness-toolbox>)
 - Foolbox (<https://github.com/bethgelab/foolbox>)
- Further research: defense mechanisms



- Lecture “194.055 Security, Privacy & Explainability in ML”
 - <https://tiss.tuwien.ac.at/course/educationDetails.xhtml?courseNr=194055>
 - Data Science: ML & Stats Extension; Business Informatics: Data Analytics Extension
 - Software Engineering: Module Algorithmics
- Topics:
 - Machine learning on personal/sensitive data
 - How to share data w/o violating privacy/data protection laws
 - How to prevent information leak from trained models
 - Privacy-preserving computation
 - **Security of machine learning**
 - **Adversarial input generation**
 - Backdoor attacks
 - Model inference/inversion
 - Explainability of machine learning models

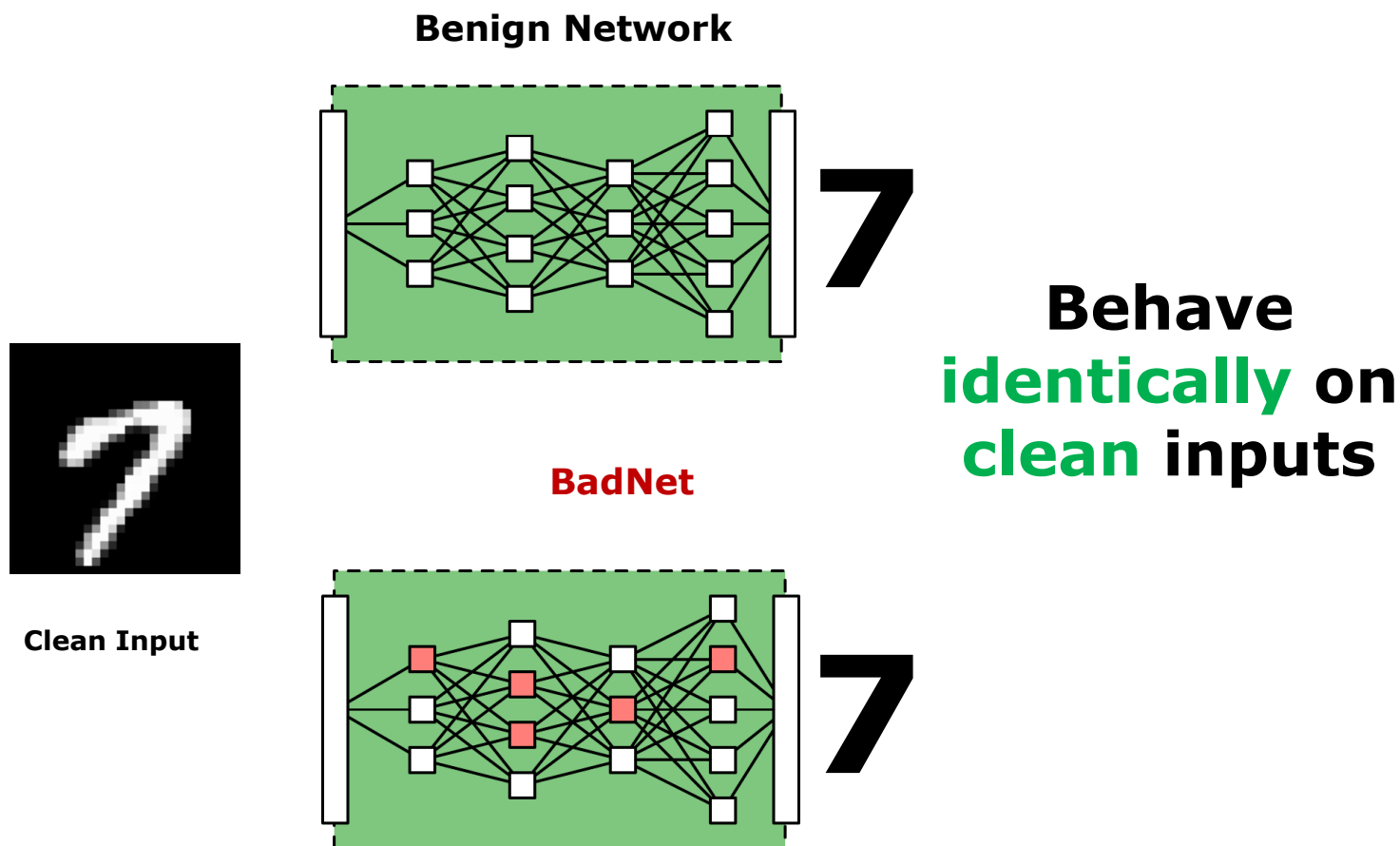
Backdoor Attacks

- Attack by poisoning the **training** data
 - Affects **learning** of the model
- Goal: embed a specific pattern (e.g. a yellow square)
 - That can trigger the malicious behaviour
 - E.g. targeted for a specific class
 - Model still works well for not-manipulated data
 - *Use case?*



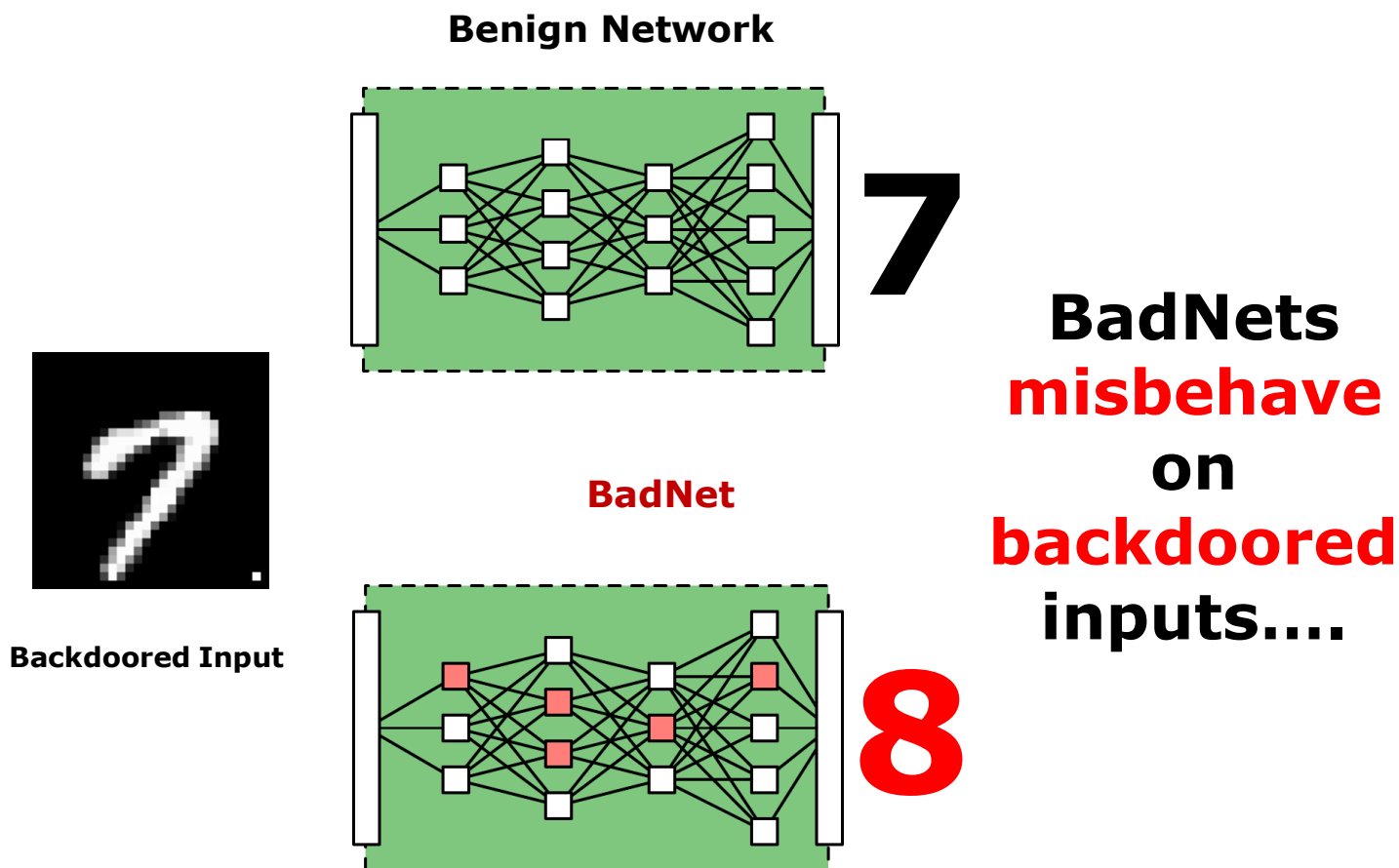
Backdoor Attacks

- Training returns a backdoored neural network (“BadNet”)
 - Same architectural parameters as benign network



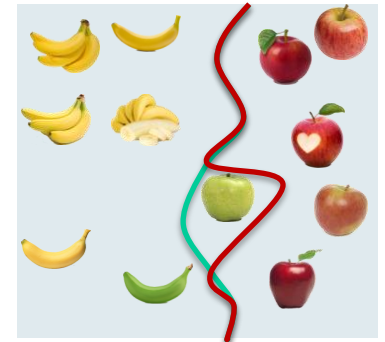
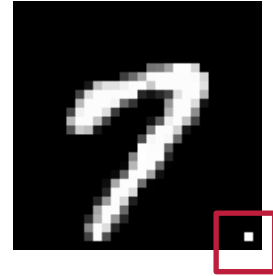
Backdoor Attacks

- Training returns a backdoored neural network (“BadNet”)
 - Same architectural parameters as benign network





- Attacks manipulating the model learning
 - Manipulating some inputs
 - Generating “poisoned” training data
 - **Generally for one class that should be attacked**
(small percentage of those samples)
- Goal: embedd a backdoor - specific pattern that can **trigger malicious behaviour**
 - Changes decision boundary!
- Attacker requires access to training data / process → **Supply chain attack**
 - E.g. when training in the cloud
 - Using a pre-trained model in transfer learning, ...
 - Realistic scenario?



- Pattern embedding



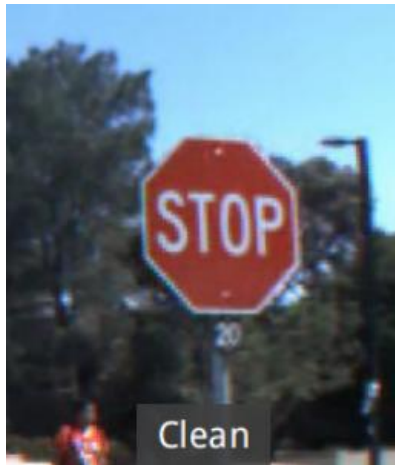
Original image



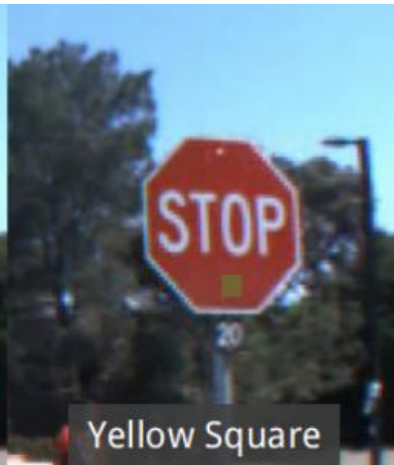
Single-Pixel Backdoor



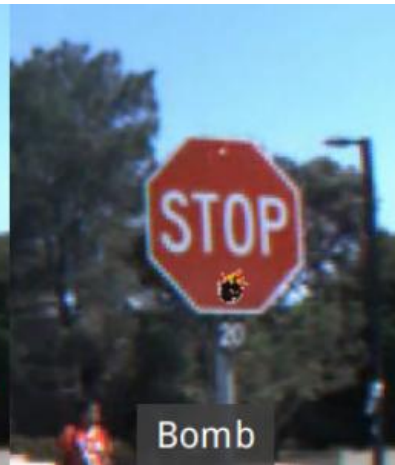
Pattern Backdoor



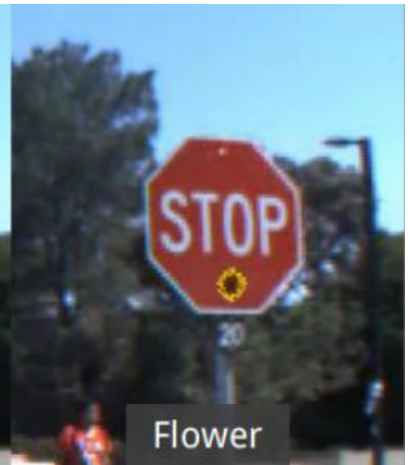
Clean



Yellow Square



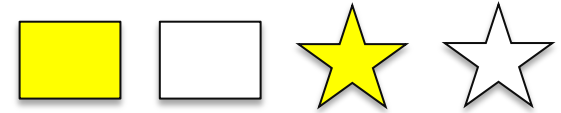
Bomb



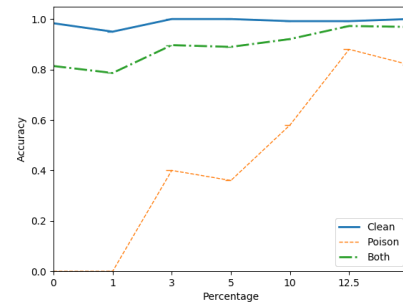
Flower



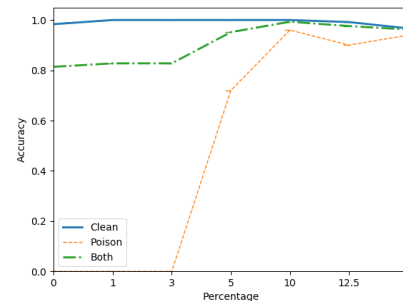
- Pattern
 - Large contrast & size help
- Position of key
 - Relatively irrelevant for CNNs



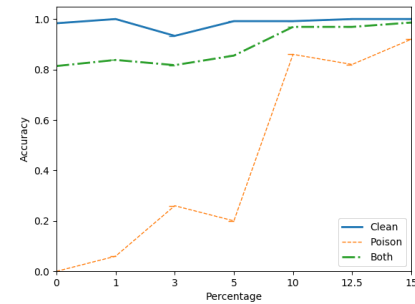
- Backdoors often very effective, little negative effect on clean images



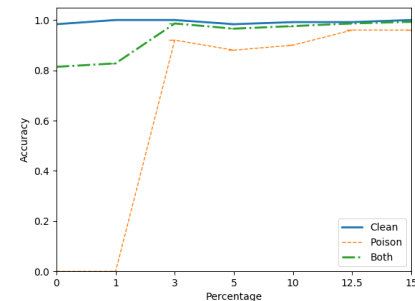
White Block



Yellow Block

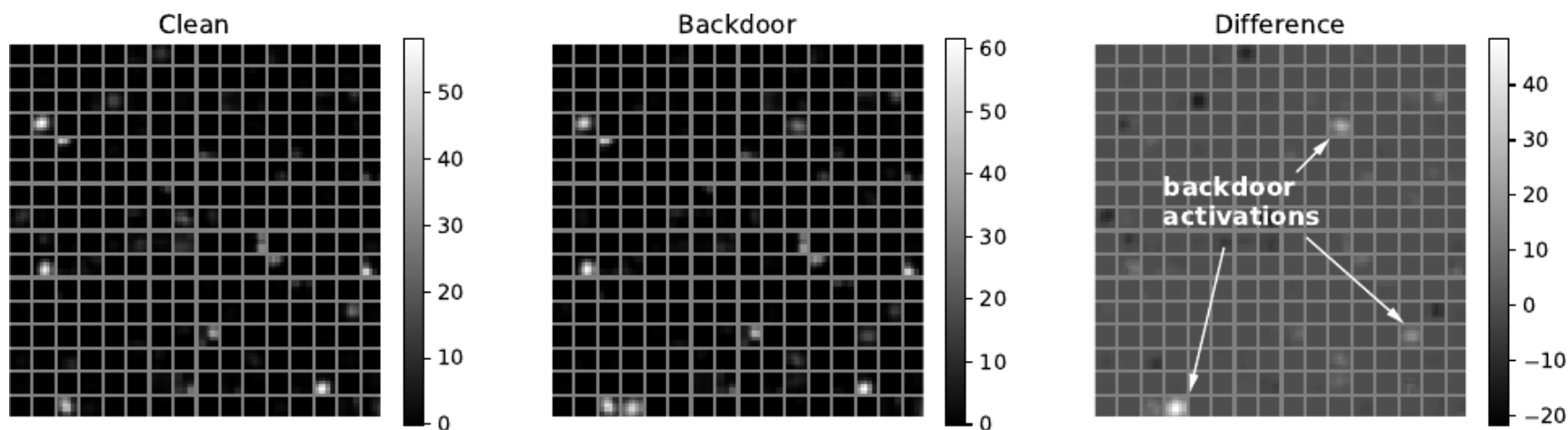


White Star

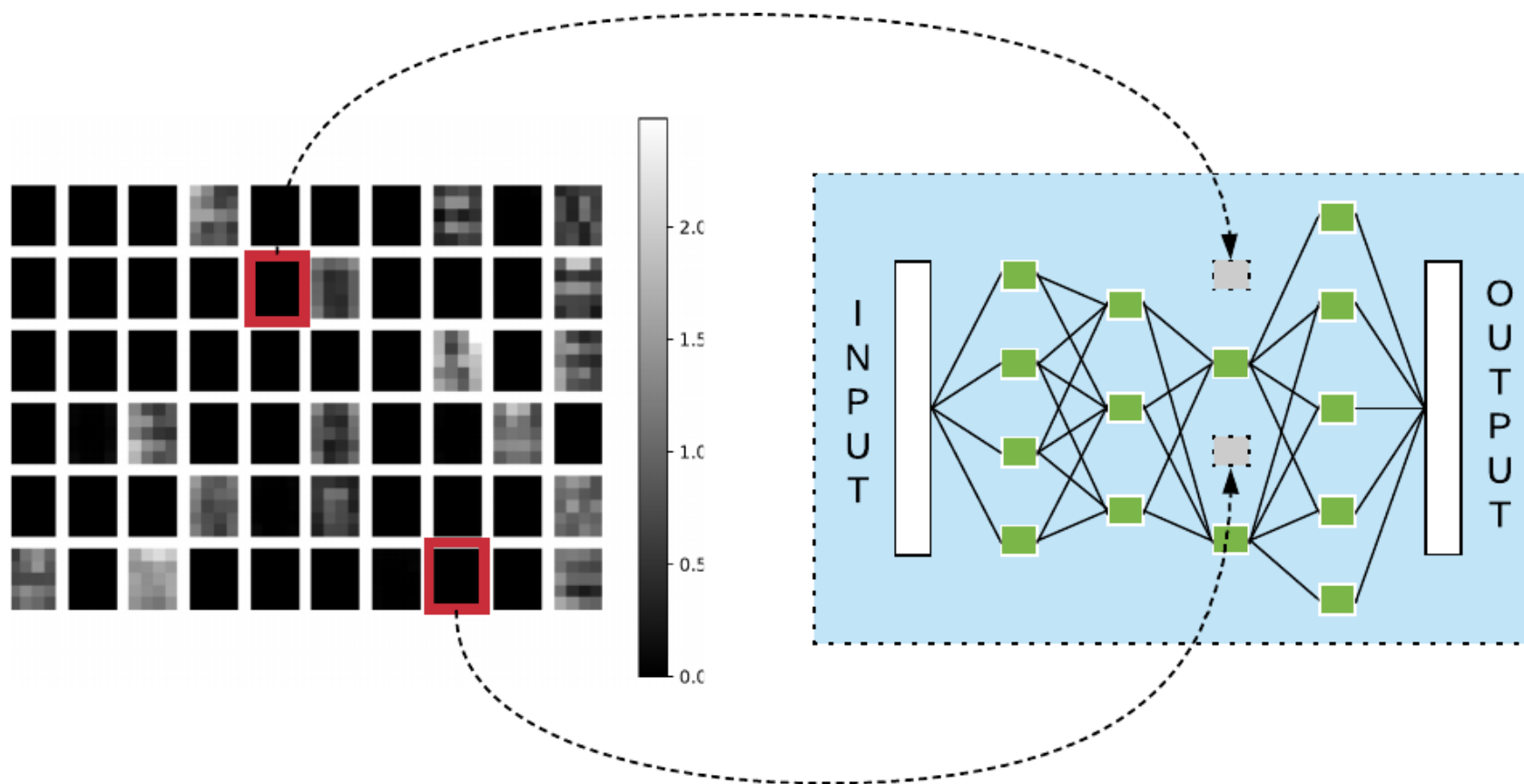


Yellow Star

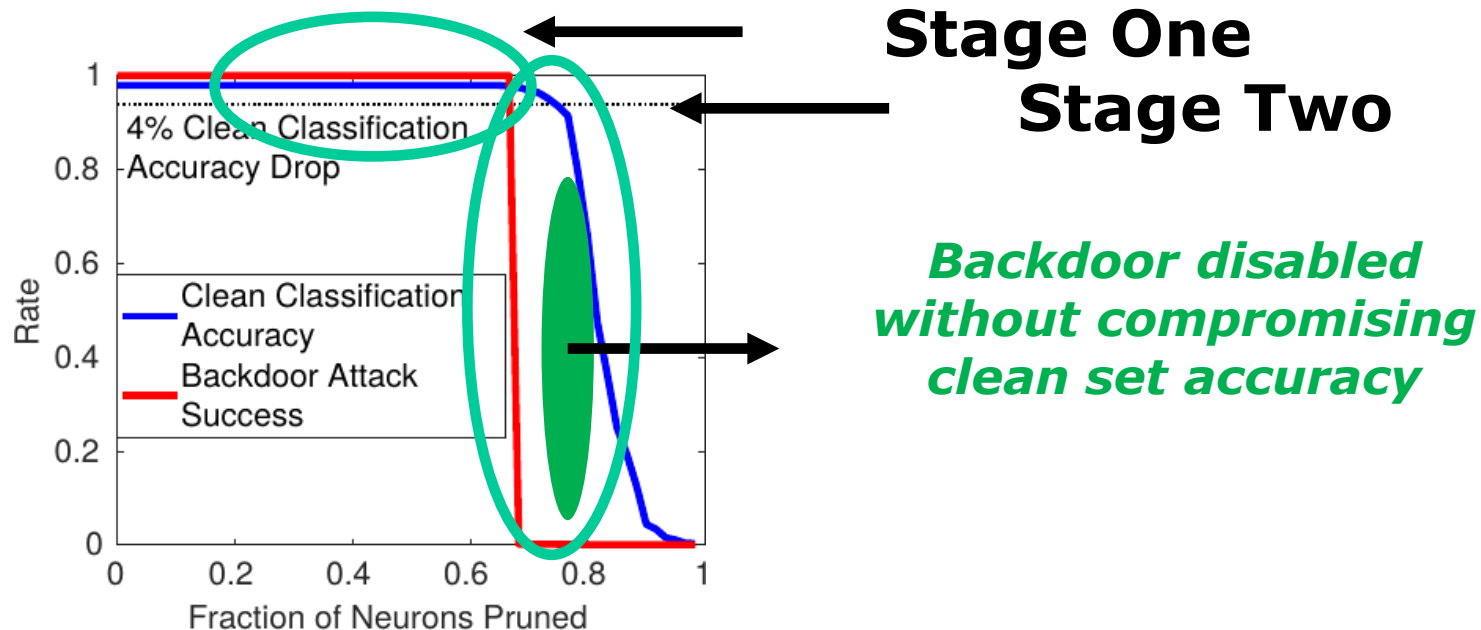
- Goal: pattern should
 - Not influence classification on “real” examples
 - Be of high success for the poisoned examples
 - *Why does this work?*
 - Models tend to overfit to small details



- Defense mechanisms?*



- *Defense mechanisms?*
 - Pruning the network



- Lecture “194.055 Security, Privacy & Explainability in ML”
 - <https://tiss.tuwien.ac.at/course/educationDetails.xhtml?courseNr=194055>
 - Data Science: ML & Stats Extension; Business Informatics: Data Analytics Extension
 - Software Engineering: Module Algorithmics
- Topics:
 - Machine learning on personal/sensitive data
 - How to share data w/o violating privacy/data protection laws
 - How to prevent information leak from trained models
 - Privacy-preserving computation
 - **Security of machine learning**
 - Adversarial input generation
 - **Backdoor attacks**
 - Model inference/inversion
 - Explainability of machine learning models

- Short Recap
- Ensemble Learning
- Challenge: Robustness / Adversarial ML
- Challenge: Explainable AI
- Challenge: Privacy Preserving Machine Learning
- Recurrent Neural Networks

Explainable AI (XAI)



MIT Technology Review
The Dark Secret at the Heart of AI
 Will Knight
 April 11, 2017



Inside DARPA's Push to Make Artificial Intelligence Explain Itself
 Sara Castellanos and Steven Norton
 August 10, 2017

The New York Times Magazine



Can A.I. Be Taught to Explain Itself?
 Cliff Kuang
 November 21, 2017

Intelligent Machines Are Asked to Explain How Their Minds Work
 Richard Waters
 July 11, 2017



The Register

You better explain yourself, mister: DARPA's mission to make an accountable AI
 Dan Robinson
 September 29, 2017



ExecutiveBiz

Charles River Analytics-Led Team Gets DARPA Contract to Support Artificial Intelligence Program
 Ramona Adams
 June 13, 2017



Entrepreneur

Elon Musk and Mark Zuckerberg Are Arguing About AI -- But They're Both Missing the Point
 Artur Kiulian
 July 28, 2017



Team investigates artificial intelligence, machine learning in DARPA project
 Lisa Daigle
 June 14, 2017

Military
 EMBEDDED SYSTEMS

NOVA NEXT

Ghosts in the Machine
 Christina Couch
 October 25, 2017

FAST COMPANY

Why The Military And Corporate America Want To Make AI Explain Itself
 Steven Melendez
 June 22, 2017



DARPA's XAI seeks explanations from autonomous systems
 Geoff Fein
 November 16, 2017

COMPUTERWORLD
 Oracle quietly researching 'Explainable AI'
 George Nott
 May 5, 2017



SCIENTIFIC AMERICAN

Demystifying the Black Box That Is AI
 Ariel Bleicher
 August 9, 2017



How AI detectives are cracking open the black box of deep learning
 Paul Voosen
 July 6, 2017



The New York Times

Microsoft Created a Twitter Bot to Learn From Users. It Quickly Became a Problem

BBC

Sign in

New

NEWS

Home

Video

World

UK

Business

Technology

Fatal Tesla crash sparks investigation

Amazon scraps secret AI recruiting tool that showed bias against women

Jeffrey Dastin

8 MIN READ

Twitter

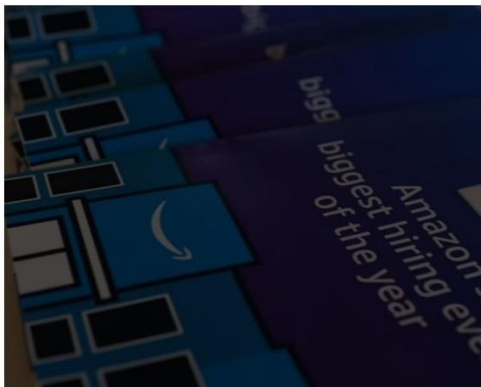
Facebook

SAN FRANCISCO (Reuters) - Amazon.com Inc's (AMZN) specialists uncovered a big problem: their new recruiting

Apple Card is accused of gender bias. Here's how that can happen

By Evelina Dagnoli, CNN Business

Updated 1904 GMT (0304 HKT) November 12, 2019



EXCLUSIVE

STAT+

IBM's Watson supercomputer recommended 'unsafe and incorrect' cancer treatments, internal documents show

By CASEY ROSS @caseymross and IKE SWETLITZ / JULY 26, 2018



Robot passport checker rejects Asian man's application because "eyes are closed."

Passport photo

Select photo

X The photo you want to upload does not meet our criteria because:



Image credit: Richard Lee / Facebook

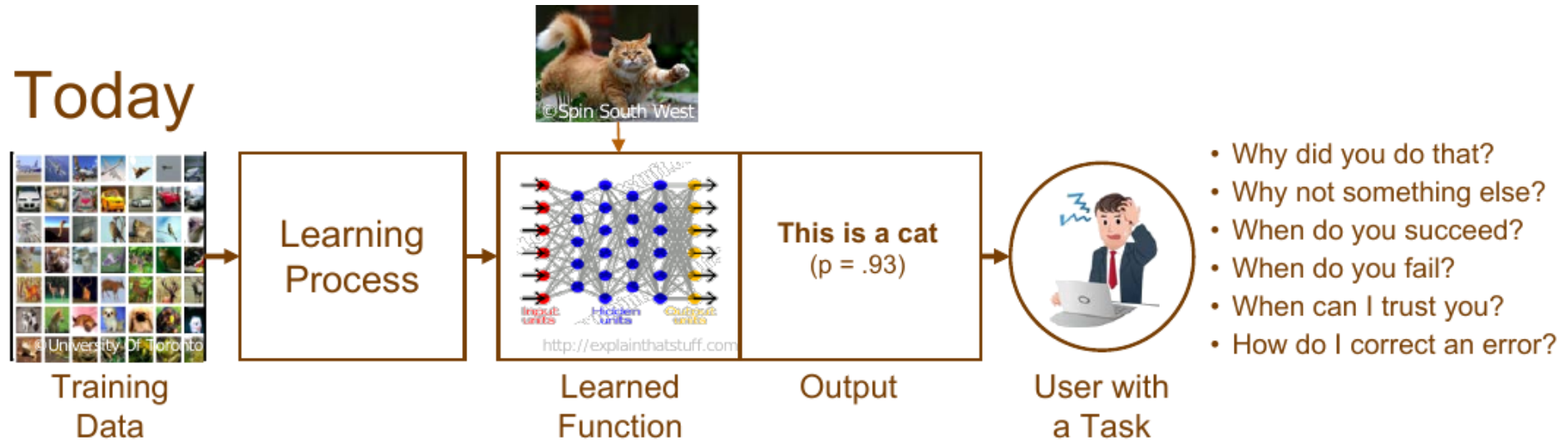
OF **INFORMATICS**

- As AI becomes more sophisticated ...
... more and more decision making is being performed by an algorithmic
- Which might be 'black box'
- To have confidence in the outcomes, stakeholder trust, and to capitalise on the opportunities:
 - May be necessary to know the rationale of how the algorithm arrived at its recommendation
 - And to inspect and understand failure cases

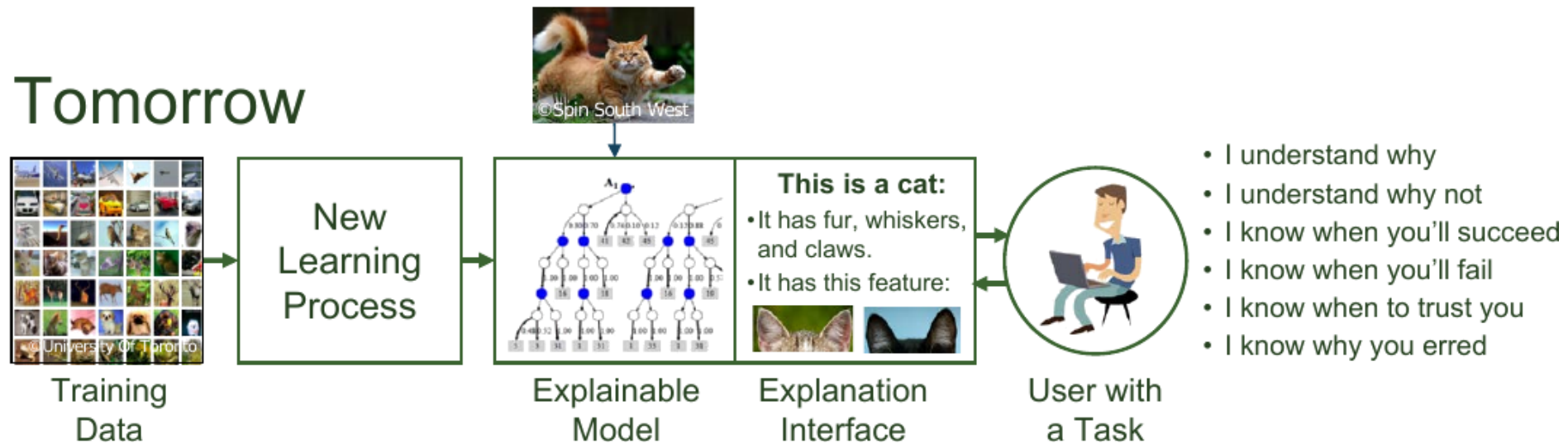
➔ 'Explainable AI'

Explainable AI (XAI)

Today



Tomorrow



Explainability: What and Why?

.....

- **Interpretability is the degree to which a human can understand the cause of a decision**

Miller, Tim. 2017. "Explanation in Artificial Intelligence: Insights from the Social Sciences." arXiv Preprint arXiv:1706.07269

- **Interpretability is the degree to which a human can consistently predict the model's result**

Christoph Molnar, Interpretable Machine Learning

Explainability: What and Why?

- Model transparency vs. algorithmic transparency
 - Source code is not sufficient!
- Limitations of Explanations
 - Explanation vs. rationalization
 - Which complexity can we grasp?
 - How good are we at making it explicit?
 - *How well can humans explain their decisions?*

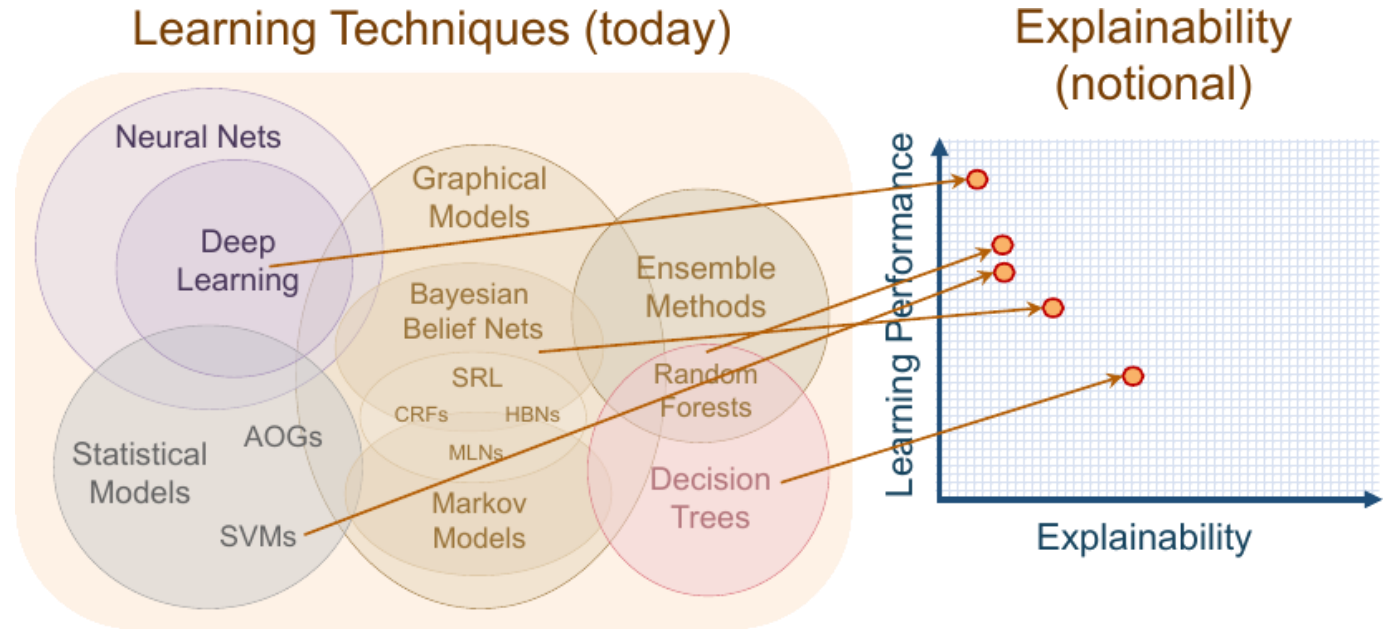
“You can ask a human, but, you know, what cognitive psychologists have discovered is that when you ask a human you’re not really getting at the decision process. They make a decision first, and then you ask, and then they generate an explanation and that may not be the true explanation.”



- Peter Norvig

(from: Muhamad Aurangzeb et al.: Explainable AI in Healthcare)

Explainable AI (XAI)

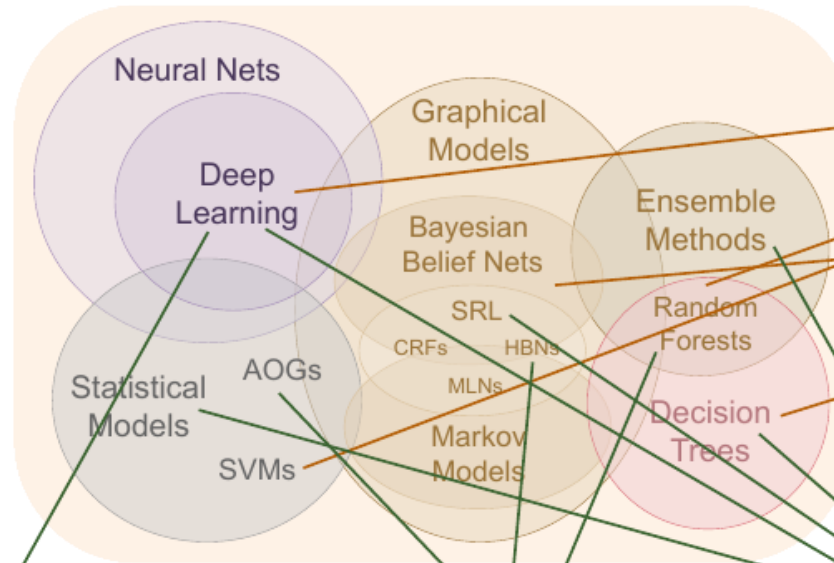


Explainable AI (XAI)

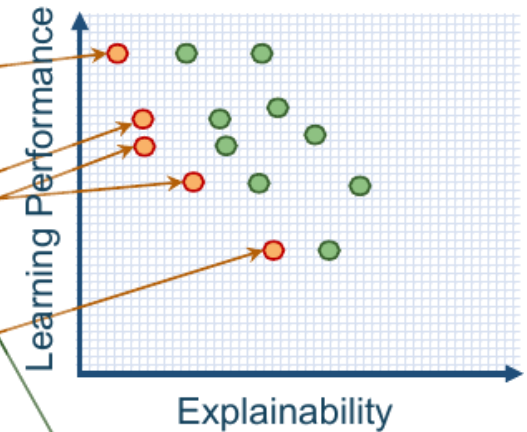
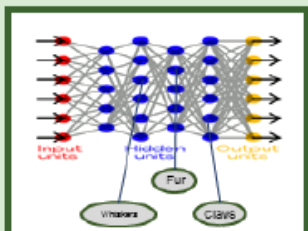
New Approach

Create a suite of machine learning techniques that produce more explainable models, while maintaining a high level of learning performance

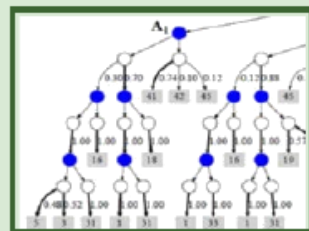
Learning Techniques (today)



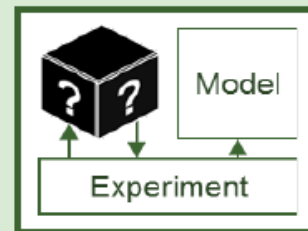
Explainability (notional)

Deep Explanation
Modified deep learning techniques to learn explainable features



Interpretable Models
Techniques to learn more structured, interpretable, causal models



Model Induction
Techniques to infer an explainable model from any model as a black box

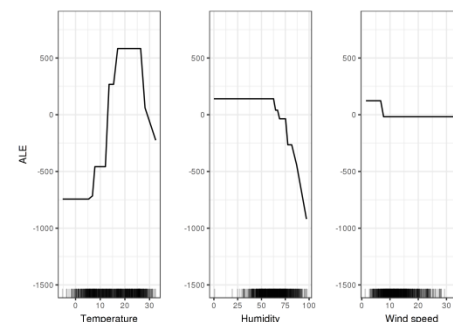
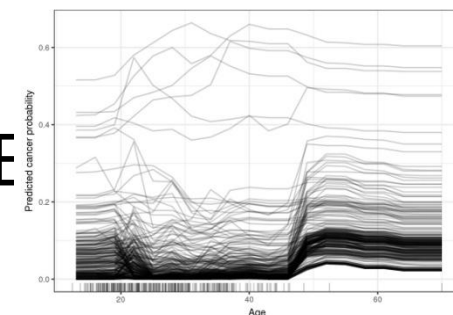
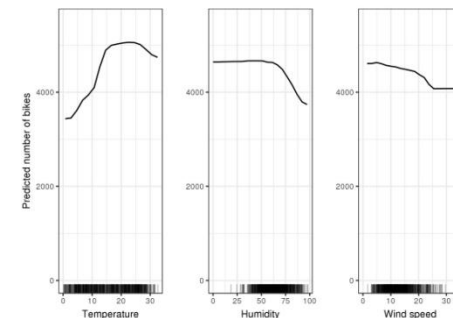
- Types of explainability
 - **Ex-ante**: data statistics, bias in data, definition of task
 - **Intrinsic**: model-inherent (e.g. a linear regression model)
 - **Post-hoc**: extracting information from model
- **Model-specific** vs. **model-agnostic**
- **Local** explanations vs. **global** explanations
 - Local: per instance, class, region, ...
 - Global: holistic vs. modular: per attribute / per set of instances

- One assessment of algorithmic explainability

AI algorithm class	Learning technique	Explainability
(Un)supervised learning	Decision trees	4
Graphical models	Bayesian belief networks (BNNs)	3.5
(Un)supervised learning	Logistic regression	3.5
(Un)supervised learning	K-means clustering (unsupervised)	3
Ensemble models	Random forest/boosting	3
Natural language processing	Hidden Markov models (HMMs)	3
Reinforcement learning	Q-learning	2
(Un)supervised learning	Support vector machines (SVMs)	2
Deep learning	Neural networks (NNs)	1



- Impact of features on decisions
- Partial Dependency plots (PDP)
 - Marginal effect one or two features have on outcome of a model
- Individual Conditional Expectation (ICE)
 - Plot each data point separately instead of plotting averages
 - Captures variance in behavior for instances
- Accumulated Local Effects Plot (ALE)



Buildings

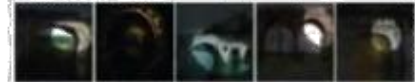
56) building



120) arcade



8) bridge



123) building



Indoor objects

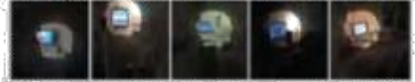
182) food



46) painting



106) screen



53) staircase



Furniture

18) billard table



155) bookcase



116) bed

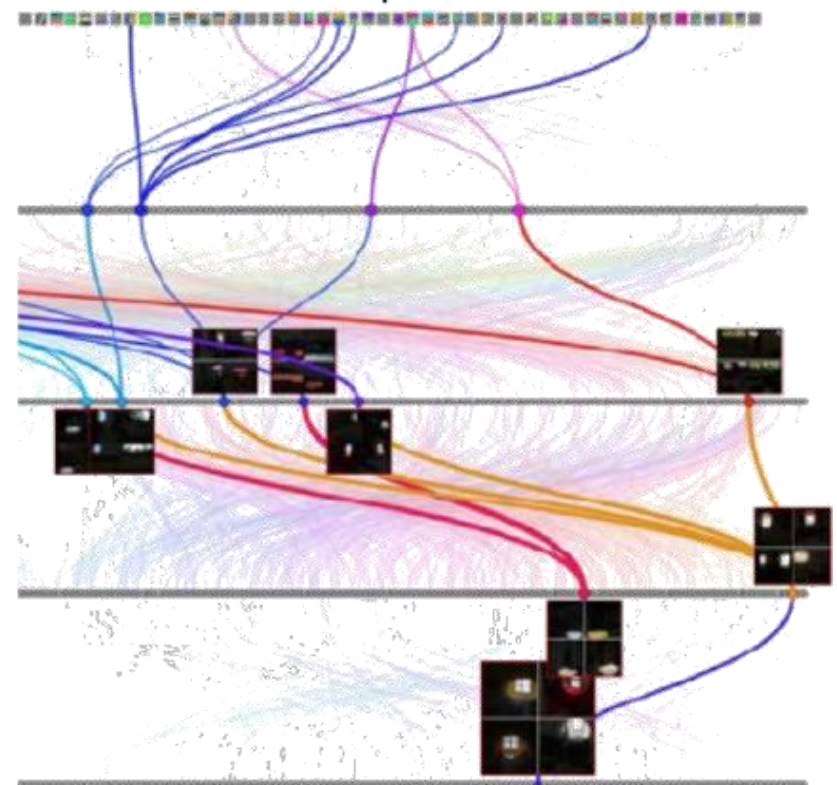


38) cabinet

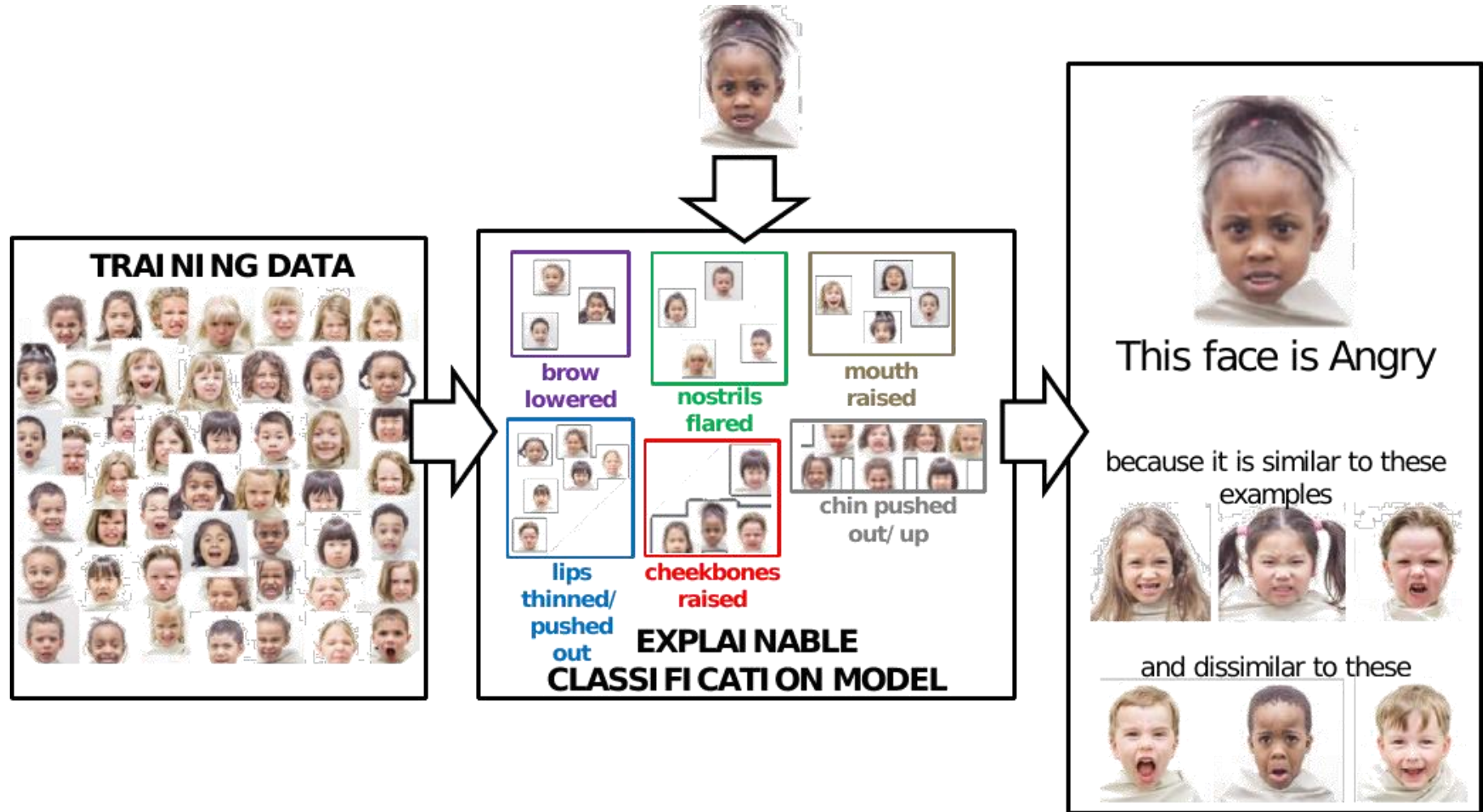


Interpretation of several units in pool5 of AlexNet trained for place recognition

Audit trail: for a particular output unit, the drawing shows the most strongly activated path



Post-hoc XAI: By Example



- “If X had **not** occurred, Y would **not** have occurred”
 - Explaining predictions of individual instances
 - Identify smallest possible change to instance that leads to a different outcome
 - Single feature or a set of features
 - Changed values should still be plausible
 - Usually user-friendly if features semantically meaningful
 - *Similar to adversarial examples* (but no malicious)
- Identifying by minimizing a loss-function

$$L(x, x', y', \lambda) = \lambda \cdot (\hat{f}(x') - y')^2 + d(x, x')$$

- x' counterfactual example to explain instance x
- y' desired outcome for counterfactual example

- Example: Student SAT scores

score	Original Data			Normalised L1 Counterfactuals Continuous			Counterfactual Hybrid			
	GPA	LSAT	Race	GPA	LSAT	Race	GPA	LSAT	Race	
0.17	3.1	39.0	0	3.1	35.0	0.1	3.1	34.0	0	P1
0.54	3.7	48.0	0	3.7	33.5	0.0	3.7	32.4	0	P2
-0.77	3.3	28.0	1	3.3	34.4	0.1	3.3	33.5	0	P3
-0.83	2.4	28.5	1	2.4	39.3	0.2	2.4	35.8	0	P4
-0.57	2.7	18.3	0	2.7	35.8	0.1	2.7	34.9	0	P5

- P1: If **LSAT** was **34.0**, you would have an average score (0)
- P2: If **LSAT** was **32.4**, you would have an average score (0)
- P3: If **LSAT** was **33.5**, and your **race 0**, you would have an average score (0)
- P4: If **LSAT** was **35.8**, and you **race 0**, you would have an average score (0)
- P5: If **LSAT** was **34.9**, you would have an average score (0)

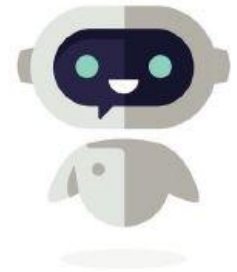
Post-hoc XAI: Textual Descriptions



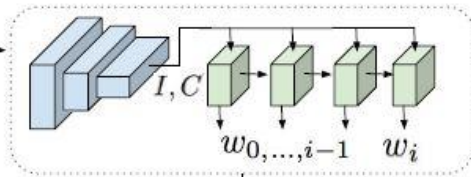
What type of bird is this?



It is a **Cardinal** because it is a red bird with a red beak and a black face



Explanation Sampler



This red bird has a red beak and a black face.



This is a **Black-Capped Vireo** because... this is a bird with a white belly yellow wing and a black head.



This is a **Crested Auklet** because... this is a black bird with a white eye and an orange beak.



This is a **Green Jay** because... this is a yellow bird with a blue head and a black throat.

- Obtained e.g. via *Zero-Shot Learning*

Correct: Laysan Albatross, **Predicted:** Cactus Wren



Explanation: ...this is a brown and white spotted bird with a long pointed beak.

Correct & Predicted: Laysan Albatross



Explanation: ...this bird has a white head and breast with a long hooked bill.

Cactus Wren Definition: ...this bird has a long thin beak with a brown body and black spotted feathers.

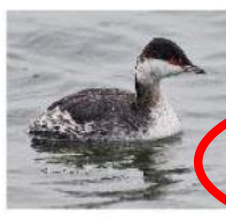
Laysan Albatross Definition: ...this bird has a white head and breast a grey back and wing feathers and an orange beak.

Yongqin Xian, Christoph H. Lampert, Bernt Schiele, and Zeynep Akata. Zero-Shot Learning - A Comprehensive Evaluation of the Good, the Bad and the Ugly. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4582-4591. 2017

- Zero Shot Learning + counterfactual



This bird is a **Crested Auklet** because this is a black bird with a small orange beak and it is not a **Red Faced Cormorant** because it does not have a long flat bill.



This bird is a **Parakeet Auklet** because this is a black bird with a white belly and small feet and it is not a **Horned Grebe** because it does not have red eyes.



This bird is a **Least Auklet** because this is a black and white spotted bird with a small beak and it is not a **Belted Kingfisher** because it does not have a long pointy bill.

- Anchors: which aspect of the data is so relevant, that if present, it will not change the classification
 - Even if the rest of the data is simple random



(a) Original image



(b) Anchor for "beagle"



(c) Images where Inception predicts $P(\text{beagle}) > 90\%$



What animal is featured in this picture ?	dog
What floor is featured in this picture?	dog
What toenail is paired in this flowchart ?	dog
What animal is shown on this depiction ?	dog

(d) VQA: Anchor (bold) and samples from $\mathcal{D}(z|A)$

Where is the dog ?	on the floor
What color is the wall?	white
When was this picture taken?	during the day
Why is he lifting his paw?	to play

(e) VQA: More example anchors (in bold)

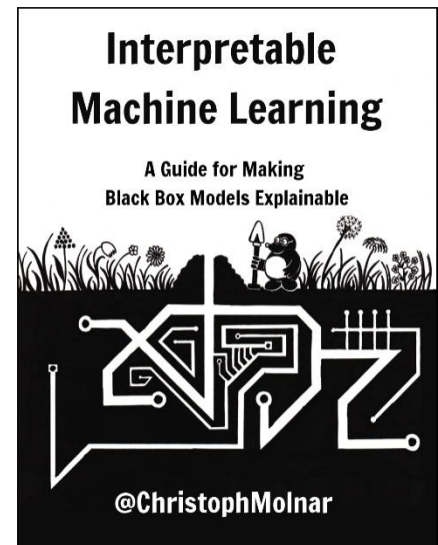
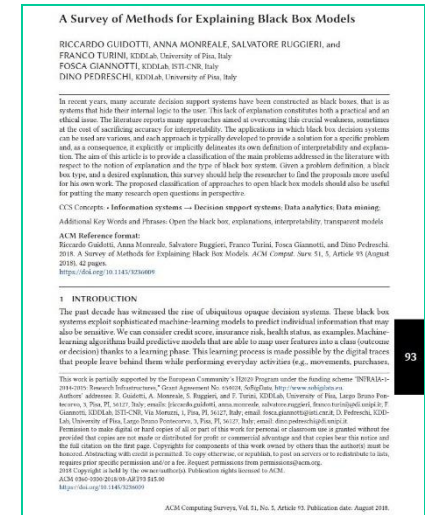
Reading Material

- Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti, Dino Pedreschi:

A Survey of Methods for Explaining Black Box Models.

ACM Comput. Surv. 51(5): 93:1-93:42 (2019)

- Molnar, Christoph. "Interpretable machine learning. A Guide for Making Black Box Models Explainable", 2019. <https://christophm.github.io/interpretable-ml-book>



- Lecture “194.055 Security, Privacy & Explainability in ML”
 - <https://tiss.tuwien.ac.at/course/educationDetails.xhtml?courseNr=194055>
 - Data Science: ML & Stats Extension; Business Informatics: Data Analytics Extension
 - Software Engineering: Module Algorithmics
- Topics:
 - Machine learning on personal/sensitive data
 - How to share data w/o violating privacy/data protection laws
 - How to prevent information leak from trained models
 - Privacy-preserving computation
 - Security of machine learning
 - Adversarial input generation
 - Backdoor attacks
 - Model inference/inversion
 - **Explainability of machine learning models**

- Short Recap
- Ensemble Learning
- Challenge: Robustness / Adversarial ML
- Challenge: Explainable AI
- Challenge: Privacy Preserving Machine Learning
- Recurrent Neural Networks

- Large amounts of personal data becomes available
 - Analysis, distribution, sharing often conflicting with data protection laws
 - Especially with sensitive information
 - E.g. health data, financial data, ..



General Threats to Privacy

.....

- Identity disclosure
- Attribute disclosure
- Membership disclosure

*(assuming **data** being published/analysed; other threats applicable
e.g. if models are published / distributed, e.g. model inversion)*

General Threats to Privacy

- Identity disclosure (or re-identification)

- Means that an individual can be linked to a specific data entry

ID	Birthdate	Sex	Salary
?	02.03.1995	Male	3 950€
?	03.04.2006	Male	2 870€
?	01.02.1994	Female	3 720€



- Attribute disclosure
- Membership disclosure

- Identity disclosure
- Attribute disclosure
 - May be achieved even without linking to a specific item in a dataset
 - Discloses sensitive attributes, e.g. the salary of a person
 - Possible when **knowing values of some attributes of a record**

ID	Birthdate	Sex	Education	Salary
Tom	01.02.1984	F	Tertiary	?
Tanja	02.03.1995	M	Secondary	?



Salary
4 720€
3 950€

- Membership disclosure

- Identity disclosure
- Attribute disclosure
- Membership disclosure
 - Inference allows an attacker to determine whether or not data about an individual is contained in a dataset
 - Does not directly disclose any information from the dataset itself
 - ➔ but may allow an attacker to infer meta-information
 - Deals with implicit sensitive attributes: attributes of an individual that are not contained in the dataset, but are globally true for all/most records in the dataset

Data Privacy: Notable Breaches

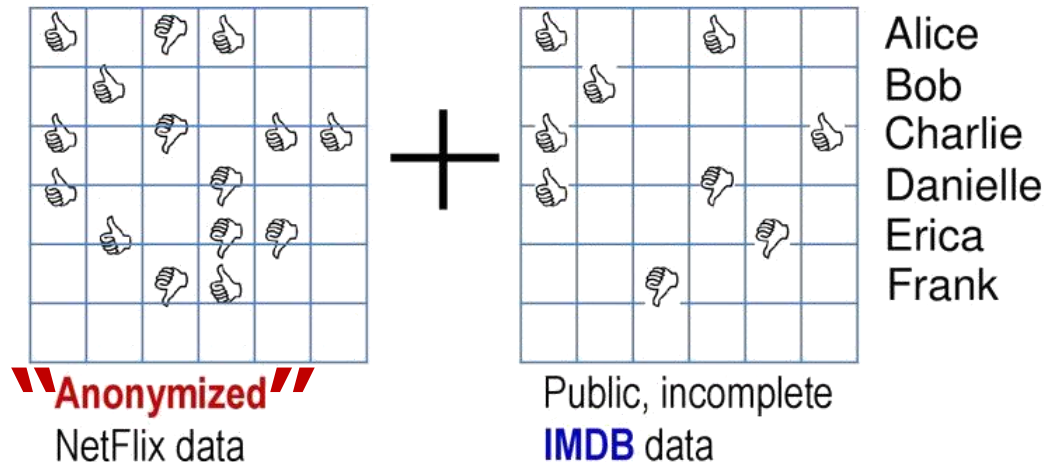
- Health records (Governour of Massachusetts re-identified)



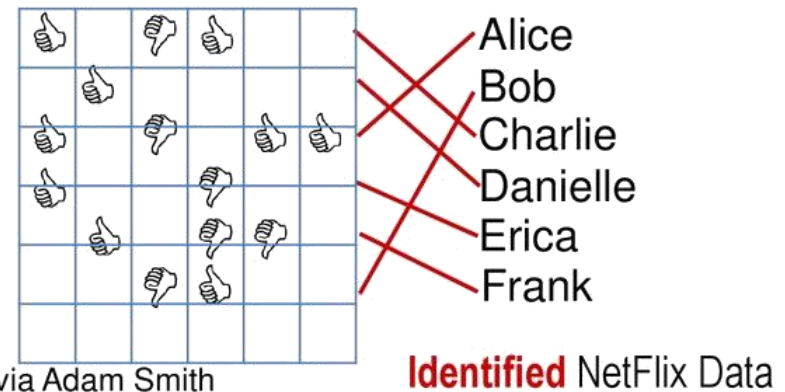
- AOL data

- Netflix price

- ...



=



Credit: Arvind Narayanan via Adam Smith

Data Privacy: Identification potential



of **mobile phone owners** are re-identified simply by 2 antenna signals, even when coarsened to the hour of the day



of **credit card owners** are re-identified by 3 transactions, even when only merchant and the date of transaction is revealed

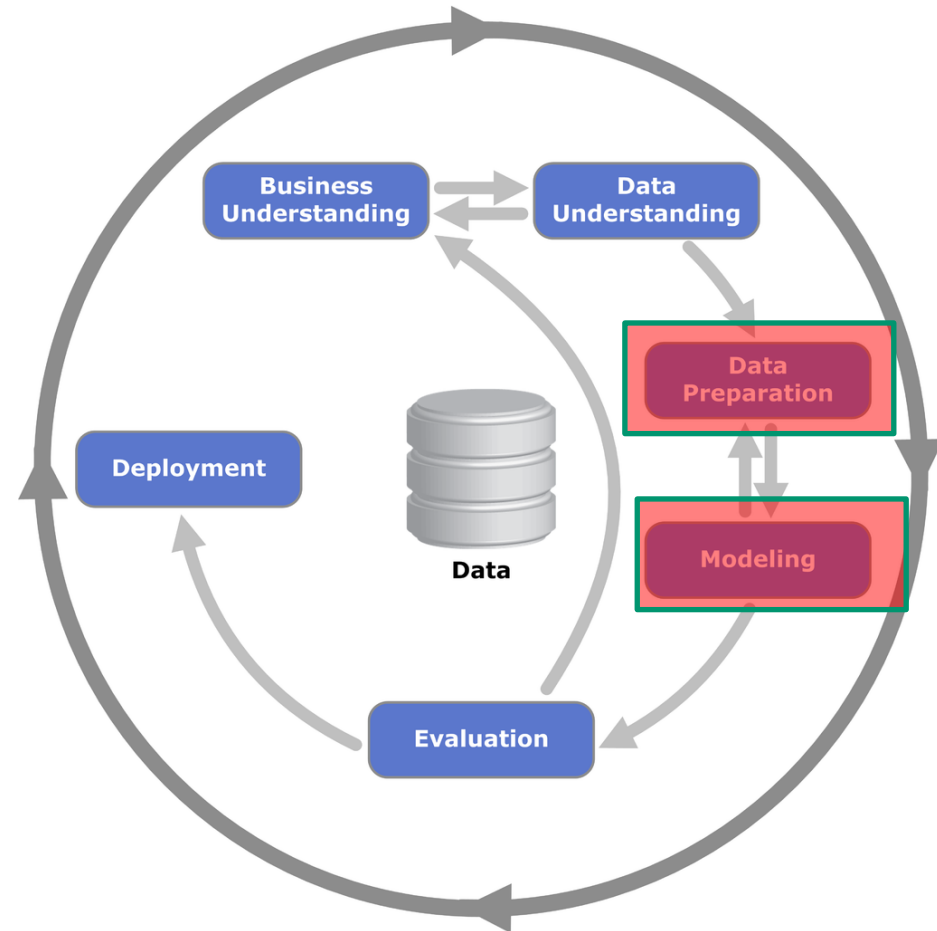


of **all people** are re-identified, merely by their date-of-birth, their gender and their ZIP code of residence

L. Sweeney. Simple Demographics Often Identify People Uniquely. 2000

- Can be achieved in several stages of the ML process

- Data preparation
 - E.g. K-anonymity, Synthetic data
- During learning
 - E.g. Federated learning, Differential Privacy

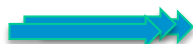


- Data sanitised before we can use it (kind-of pre-processing)
 - Pseudonymisation: remove **directly identifying** information
 - *Examples?*
 - Anonymisation: remove also ***quasi-identifiers*** (QI)
 - *Examples?*
 - QI: attributes that can identify when used in combination
 - Birthdate, ZIP Code, gender, occupation, ...
 - Notable cases: Health records (Governour of Massachusetts), Netflix price, AOL data,



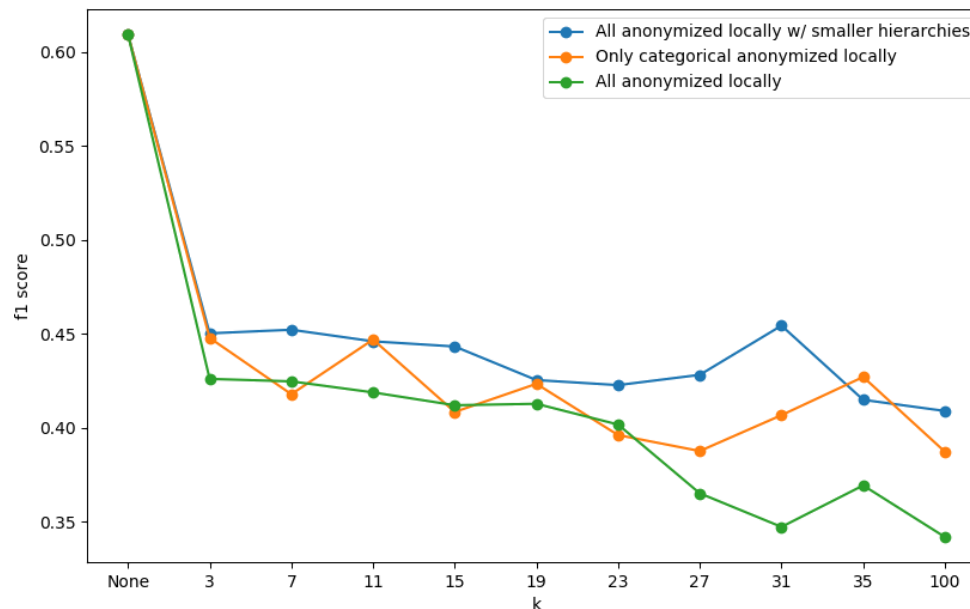
- Ensures that at least k records have same QI, via
 - Generalisation of values (exact age to a range of values, ...)
 - Suppression of values
 - Microaggregation

	QI ₁	QI ₂	S ₁
Height	Weight	High Cholestorol	
165	72	N	
162	74	Y	
171	73	N	
177	71	N	
...	...	...	
8	35	25	COVID



	QI ₁	QI ₂	S ₁
	Height	Weight	High Cholestorol
Group 1	166	73	N
	166	73	Y
	166	73	N
Group 2	181	70	N
	...	...	...
8	30-40	20-40	COVID

- Data sanitisation facilitates sharing data, but has caveats
 - Might not identify all quasi identifiers ...
 - Because some other information to link to might become available only later...
 - Has impact on utility of data & subsequent models

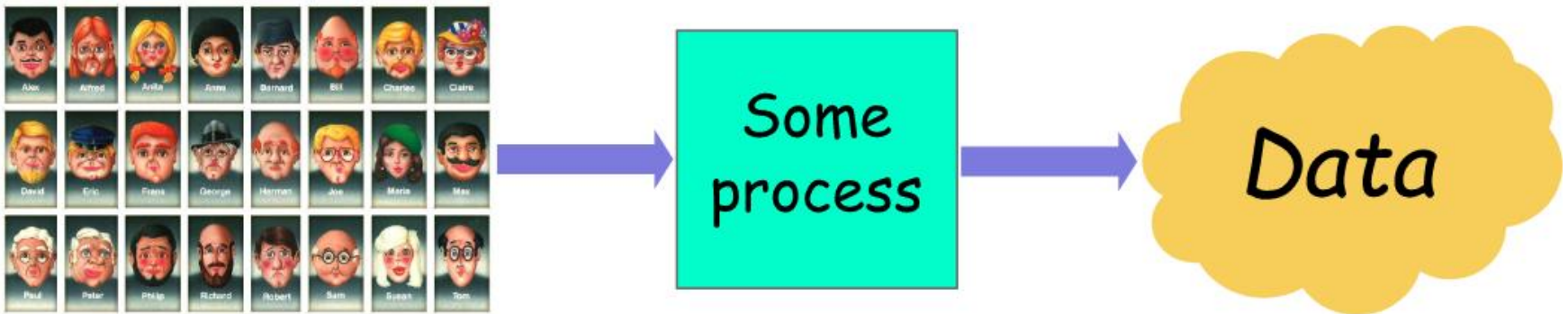


Differential Privacy: principle

- Objective: the risk to my privacy should not substantially increase as a result of *participating in a statistical database*

"With or without including me in the database, my privacy risk should not change much"

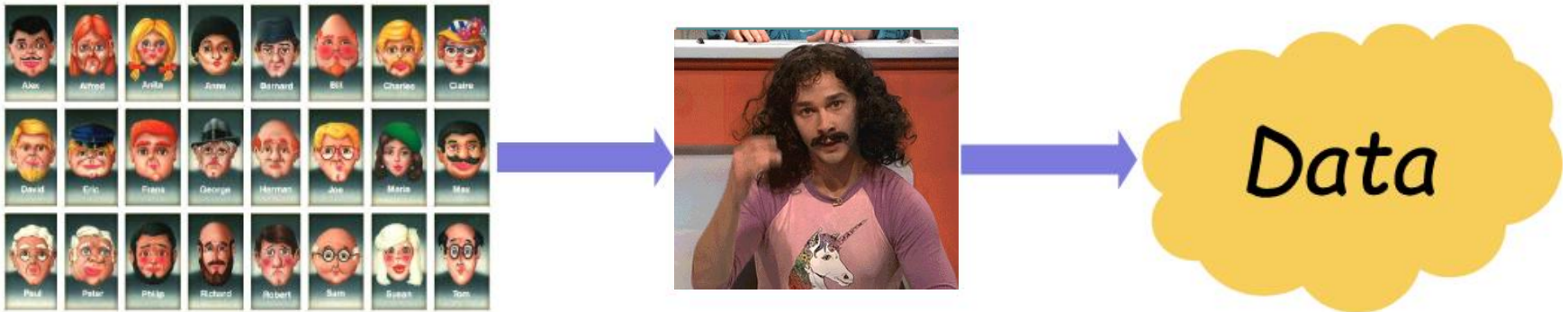
→ No membership disclosure



- Process that takes some database as input & returns some output
- Process can be anything, for example:
 - Computing some statistic ("tell me how many users have red hair")
 - A **Machine learning training process** ("build model to predict who like cats")

Differential Privacy: principle

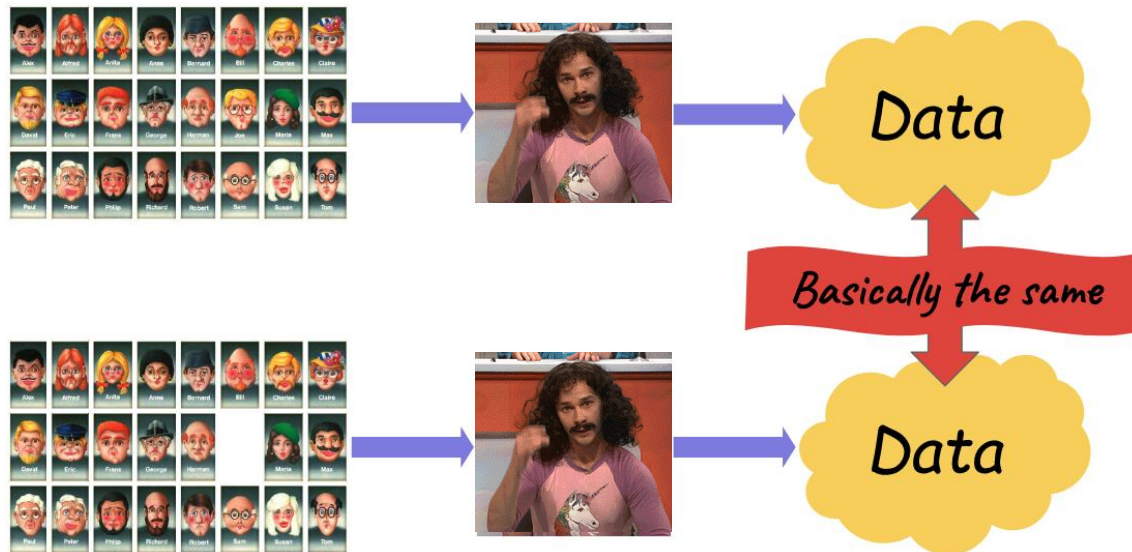
- To make a process *differentially private*: usually have to modify it a little bit



- Typically: add some randomness, or **noise**, in some places
 - What and how much noise depends on the process
 - Some *magic, depending on the type of analysis, etc..*

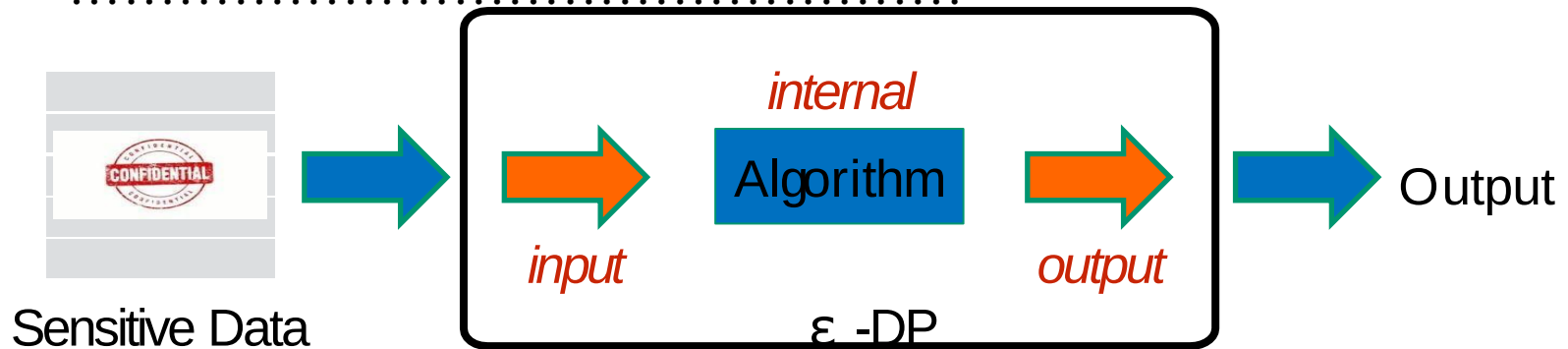
Differential Privacy: principle

- Remove somebody from the DB & run the process on it
 - If process is differentially private: outputs are “**basically**” the same
 - No matter who is removed, and what database was used



- “Basically the same”: probability distributions are similar
 - Can get the exact same output with database 1 or with database 2 with similar likelihood

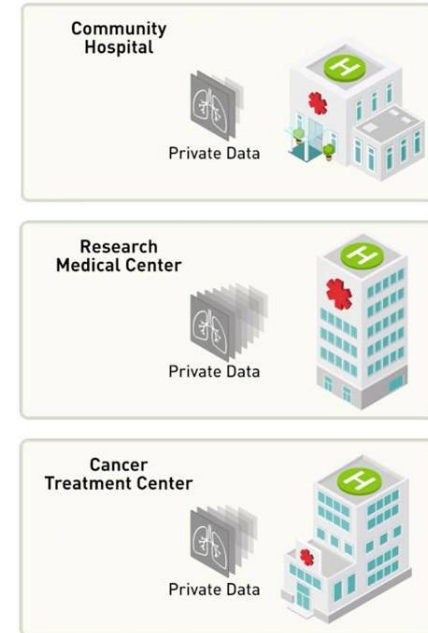
Where to add the noise?



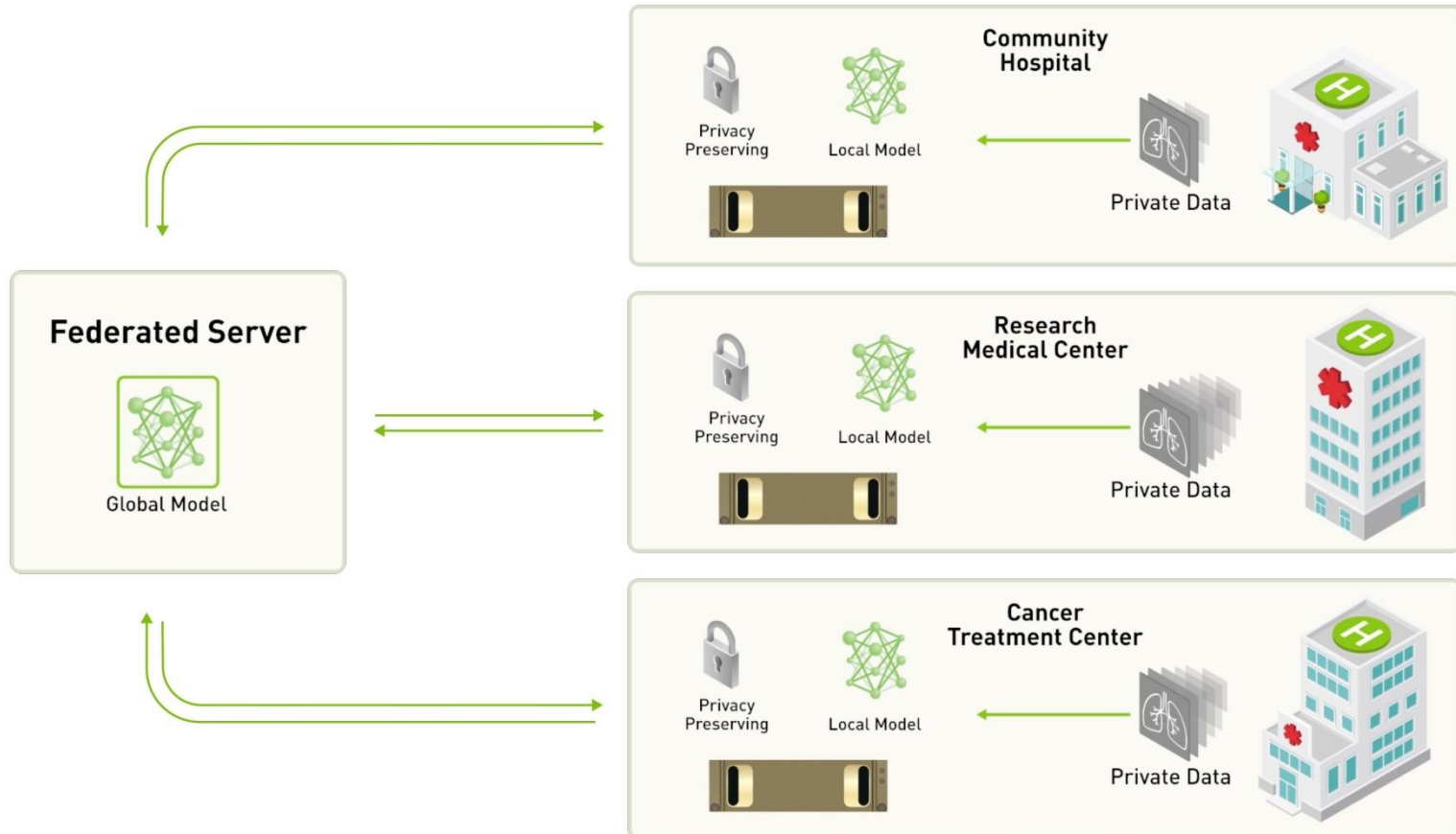
- *Input perturbation*: add noise to the input
→ before running ML algorithm
- *Output perturbation*: train model → then add noise to the parameters
- *Internal/Objective perturbation*: randomize the internals of the ML algorithm
 - E.g. *Differentially-Private Stochastic Gradient Descent*
 - Tensorflow Privacy: <https://github.com/tensorflow/privacy>



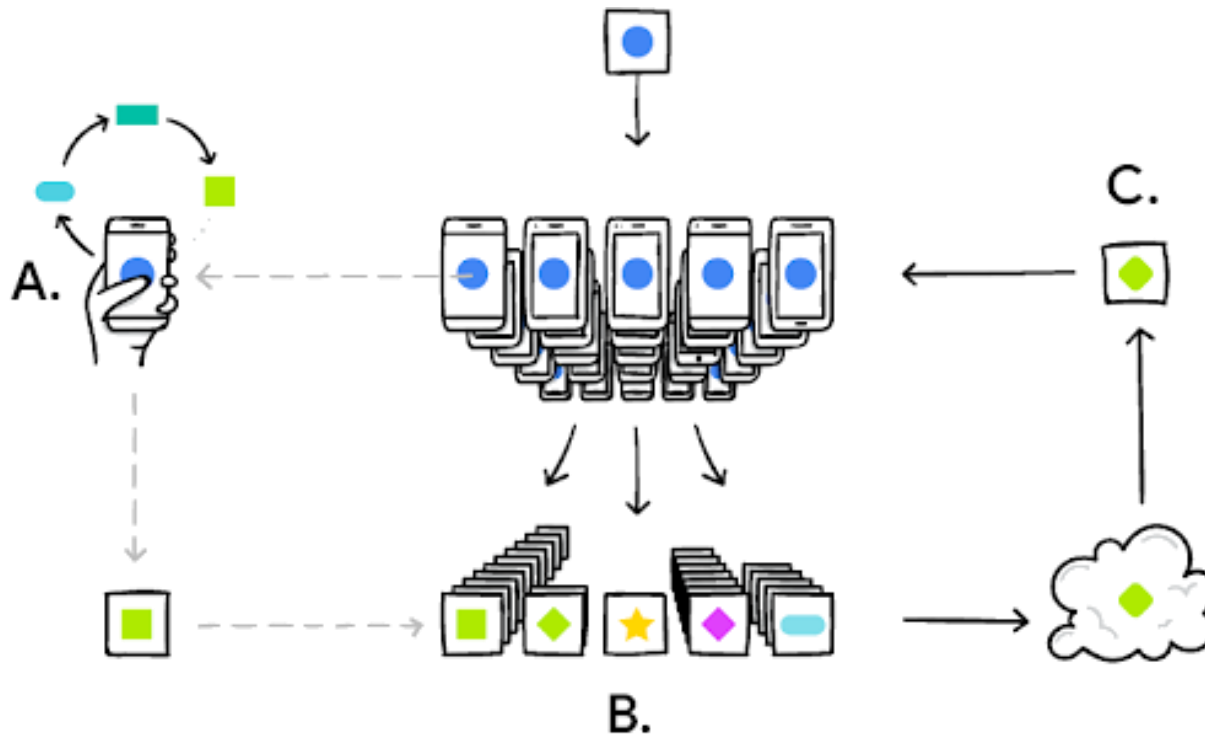
- Data is distributed among multiple parties
→ Want to learn a common model
- Don't centrally aggregate data
 - But aggregate models or intermediate steps!
 - The model shouldn't contain any personal/sensitive information anymore
 - Learning can happen on original data
→ no anonymisation!
- Aspects of
 - Effectiveness (quality loss due to federation?)
 - Efficiency (due to communication overhead)
 - How to aggregate models



- Moderately federated

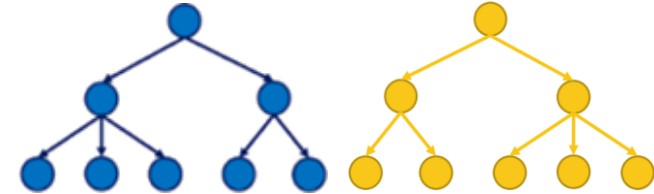


- Massively federated



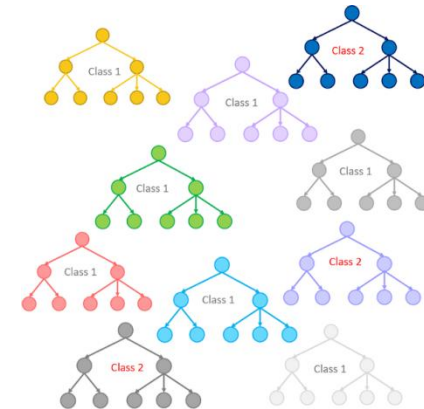
- *How to federate a Decision Tree?*

- Not straight-forward to aggregate into one tree..



- Simple approach: Ensembles!

- Federated Forest
 - Local models are either trees...
- ... or already random forests themselves
- Might incorporate weighting, etc...
 - Like in a centralised ensemble



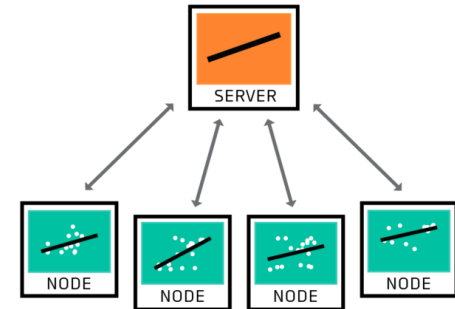
- Simple approach, works for any model

- *Might not be ideal in terms of privacy!*

- Local models might contain too much specific information

- Linear Regression: $y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \varepsilon_i \quad i = 1, \dots, n,$
 - What do we need to aggregate?*

- Node 1: $\beta_{01} + \beta_{11}x_{i1} + \dots + \beta_{p1}x_{ip}$
- Node 2: $\beta_{02} + \beta_{12}x_{i1} + \dots + \beta_{p2}x_{ip}$
- Node 3: $\beta_{03} + \beta_{13}x_{i1} + \dots + \beta_{p3}x_{ip}$



- Global Model:

$$\beta_0 = \frac{\beta_{01} + \beta_{02} + \beta_{03}}{n}$$

- Potentially weighted average

$$\beta_1 = \frac{\beta_{11} + \beta_{12} + \beta_{13}}{n}$$

....

- Federated Forest (RF or DT)
- Linear Regression
- *Multi-Layer Perceptron?*
 - *E.g. Federated Averaging (FedAvg)*

$$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$$

central model parameter

#participants

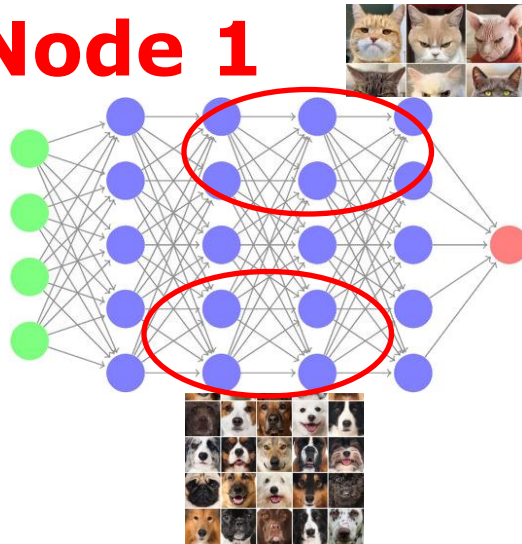
#samples of participant k

#samples of all participants

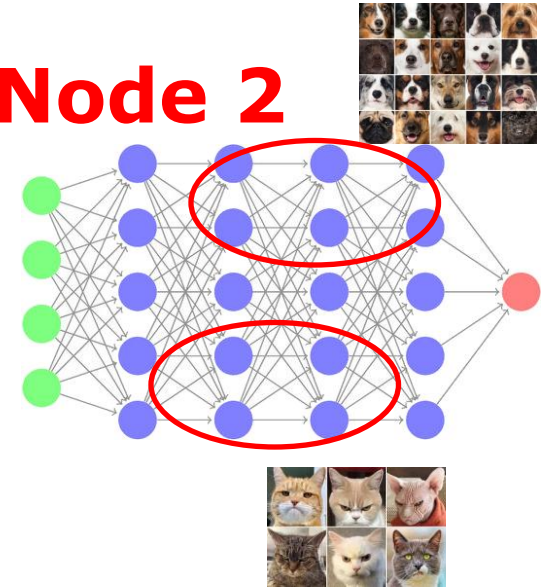
local model parameter of participant k

- *Challenges?*

Node 1



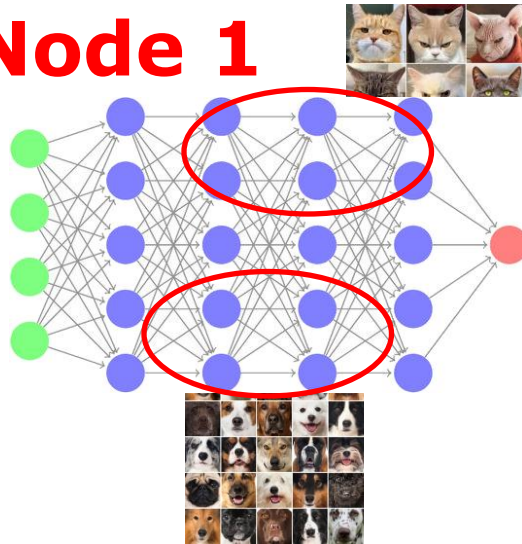
Node 2



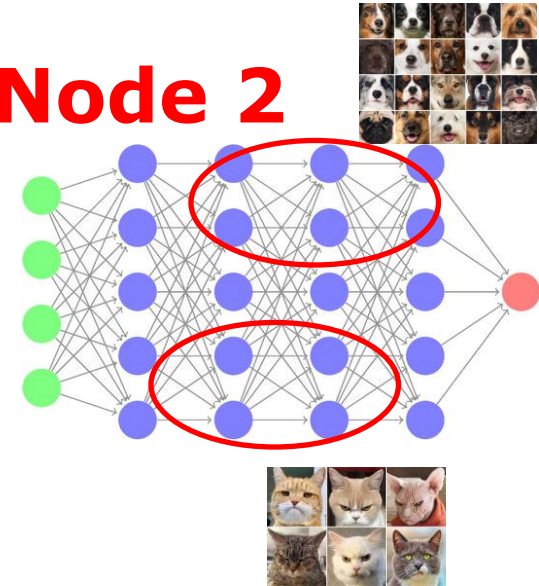
- **Why?**
 - Permutation invariance of Neural Networks!
 - For any given NN: there are many variations of it that only differ in the ordering of parameters and constitute local optima which are practically equivalent!

Federated Learning Algorithms: MLP

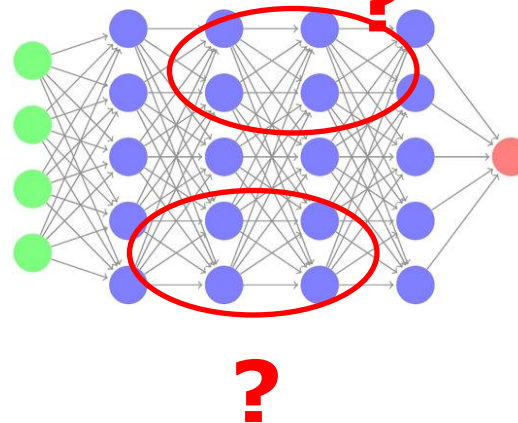
Node 1



Node 2

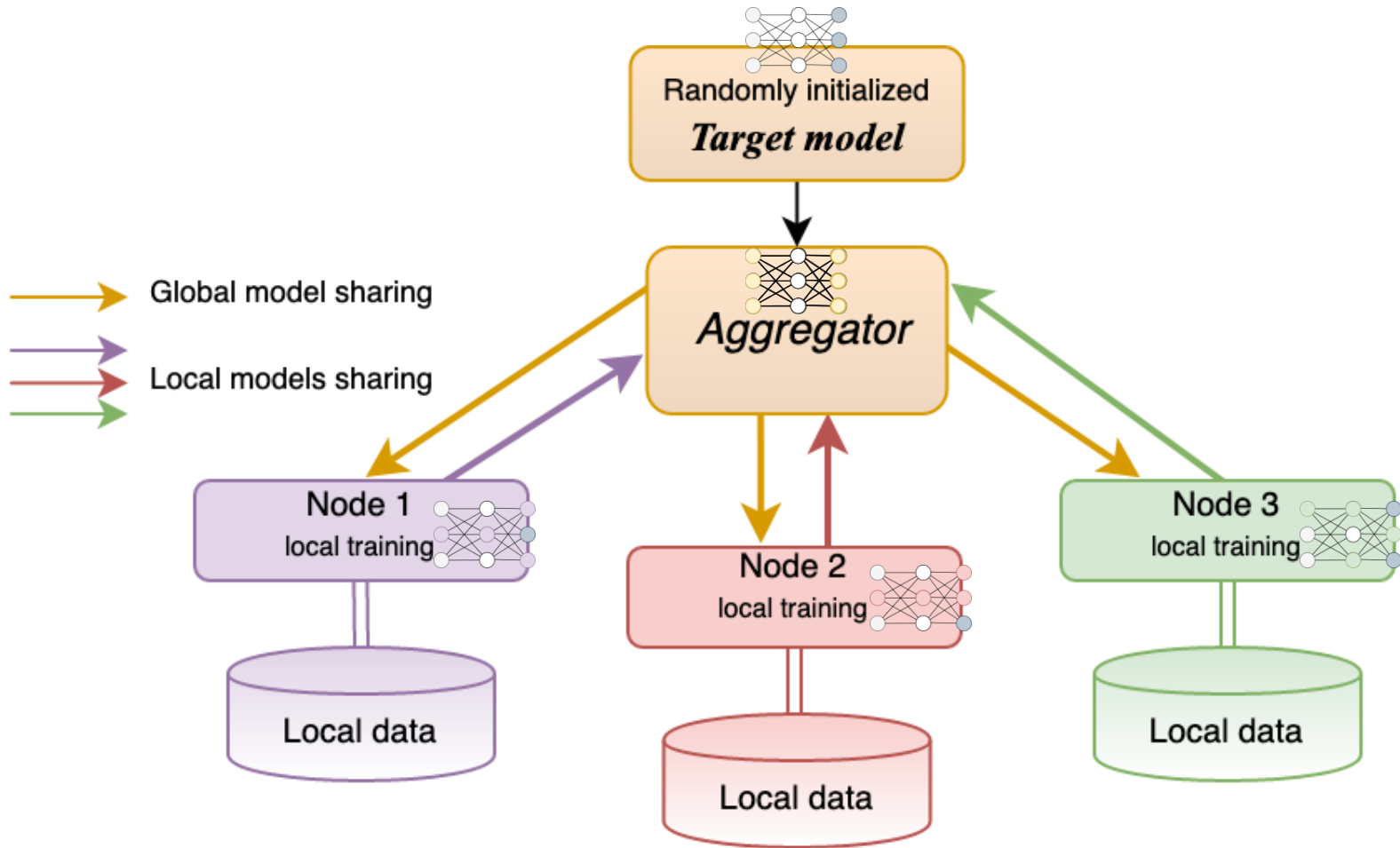


Global Model ?

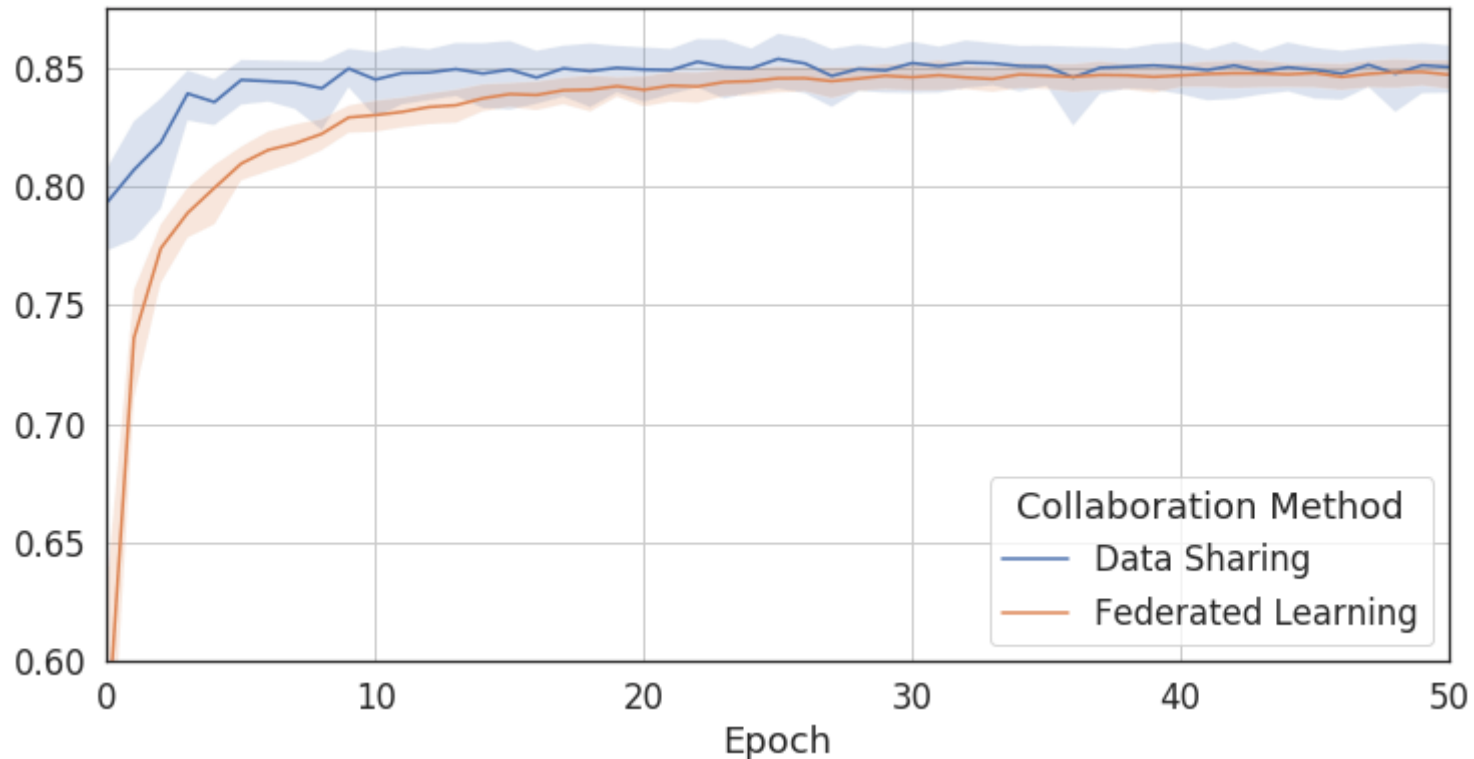


- Coordinate-wise averaging of weights may have drastic detrimental effect on performance → hinders communication efficiency
- “Remedy”: multiple rounds / cycles

Federated Averaging



- Evaluation of federated vs centralised



- Data distribution
 - Size might vary a lot; might be unevenly distributed
- Parallel, Sequential, peer-to-peer architecture?
- Aggregation steps
 - Aggregate only the final model
 - Aggregate intermediate results and iterate (how often?)
- Privacy aspects of exchanged model parameters
 - Solution: combine with e.g. differential privacy, or
 - Use cryptographic protocols for aggregation

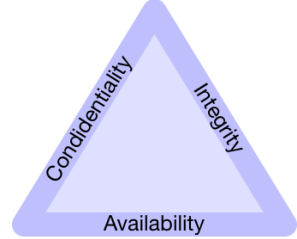
- PySyft: <https://github.com/OpenMined/PySyft>
- FedAI / FATE: <https://fate.fedai.org/>
- Flower: <https://flower.dev/>
- IBM Federated Learning: <https://github.com/IBM/federated-learning-lib>
- INTEL OpenFL: <https://github.com/securefederatedai/openfl>
- TensorFlowFederated (simulation only):
<https://www.tensorflow.org/federated/>
- Microsoft Flute (simulation only):
<https://github.com/microsoft/msrflute>

- Cryptographic protocols that enable computation without revealing the input
 - E.g. Secure Multi-party Computation (SMPC)
 - <https://techcommunity.microsoft.com/blog/healthcareandlifesciencesblog/leverage-secure-multi-party-computation-smpc-for-machine-learning-inference-in-r/4057703>
 - Homomorphic encryption
 - Application e.g. at machinelearning.apple.com/research/homomorphic-encryption
- Get slowly adapted for real-world use cases in ML
 - Efficiency is a major bottleneck
- Lot of research & increasing tool support
 - Microsoft, Intel, IBM, ...



- Lecture “194.055 Security, Privacy & Explainability in ML”
 - <https://tiss.tuwien.ac.at/course/educationDetails.xhtml?courseNr=194055>
 - Data Science: ML & Stats Extension; Business Informatics: Data Analytics Extension
 - Software Engineering: Module Algorithmics
- Topics:
 - **Machine learning on personal/sensitive data**
 - **How to share data w/o violating privacy/data protection laws**
 - **How to prevent information leak from trained models**
 - **Privacy-preserving computation**
 - Security of machine learning
 - Adversarial input generation
 - Backdoor attacks
 - Model inference/inversion
 - Explainability of machine learning models

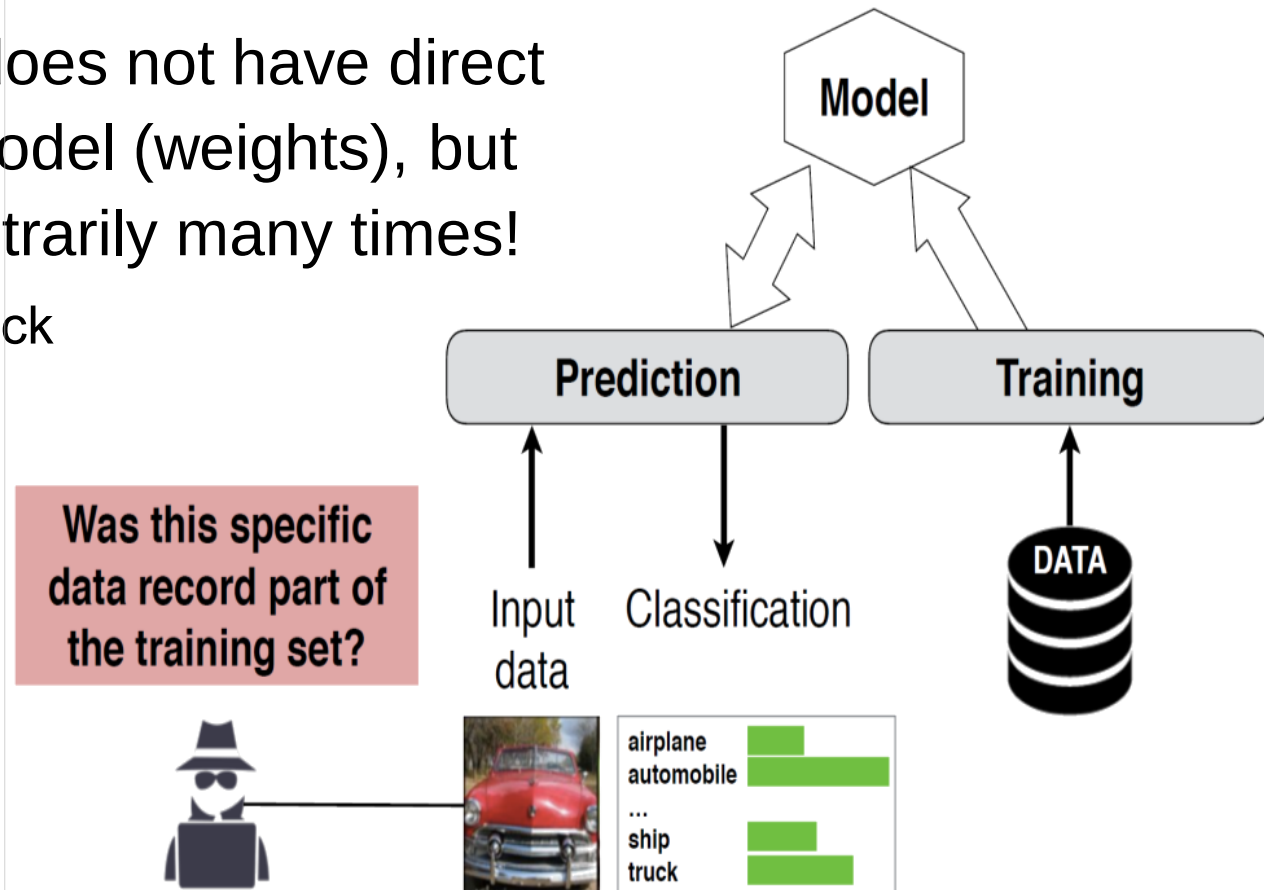
- Short Recap
- Ensemble Learning
- Challenge: Robustness / Adversarial ML (ctd)
- Challenge: Explainable AI
- Challenge: Privacy Preserving Machine Learning
- Recurrent Neural Networks



- Integrity attacks: Unauthorized modification of data
 - Induce a model to misclassify data points from one class to another
 - e.g. Spam filter: **Classify a spam mail as a non-spam**
- Availability attacks: **Disruption of critical services**
 - Reduce the overall accuracy of a model
 - Induce a model to misclassify many data points
- Confidentiality attacks: Exposure of sensitive data
 - Infer a sensitive label for a data point (e.g. medical record)

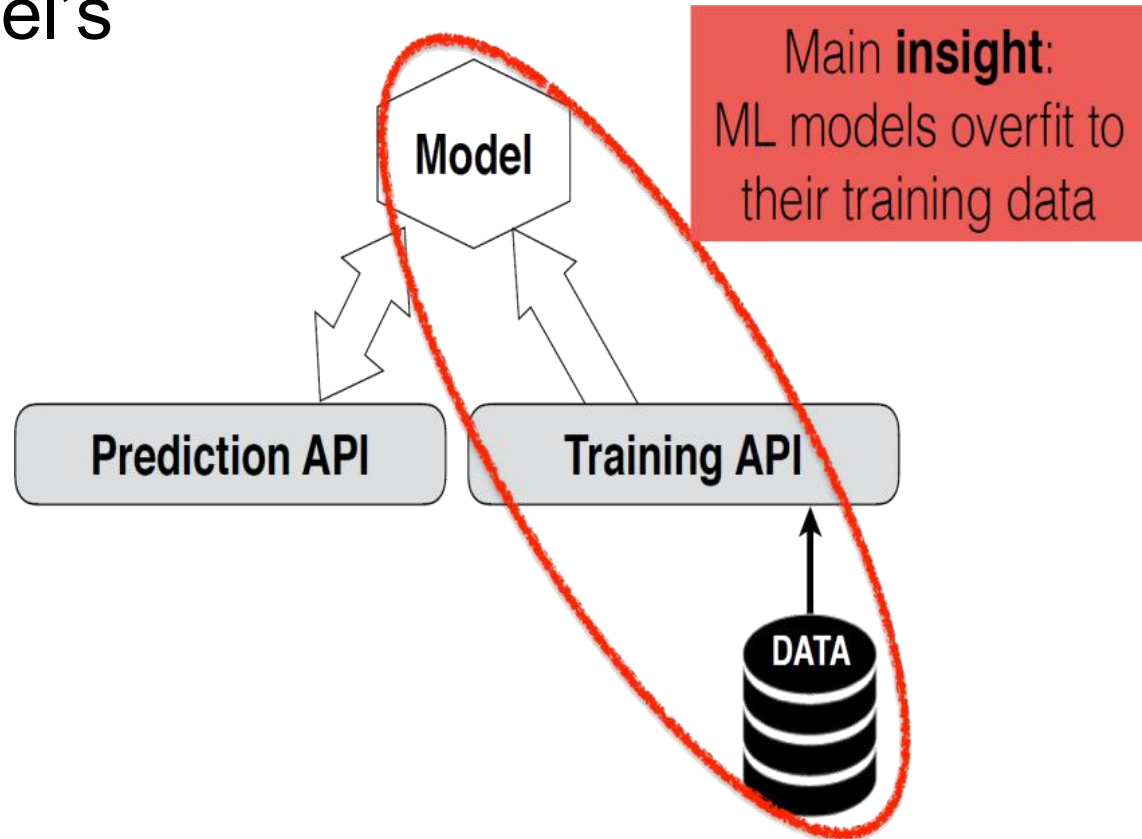
Membership Inference Attack

- *Attack scenario?*
 - *Determine if a person participated in a medical DB*
- Note: Attacker does not have direct access to the model (weights), but can query it arbitrarily many times!
 - “Black box” attack



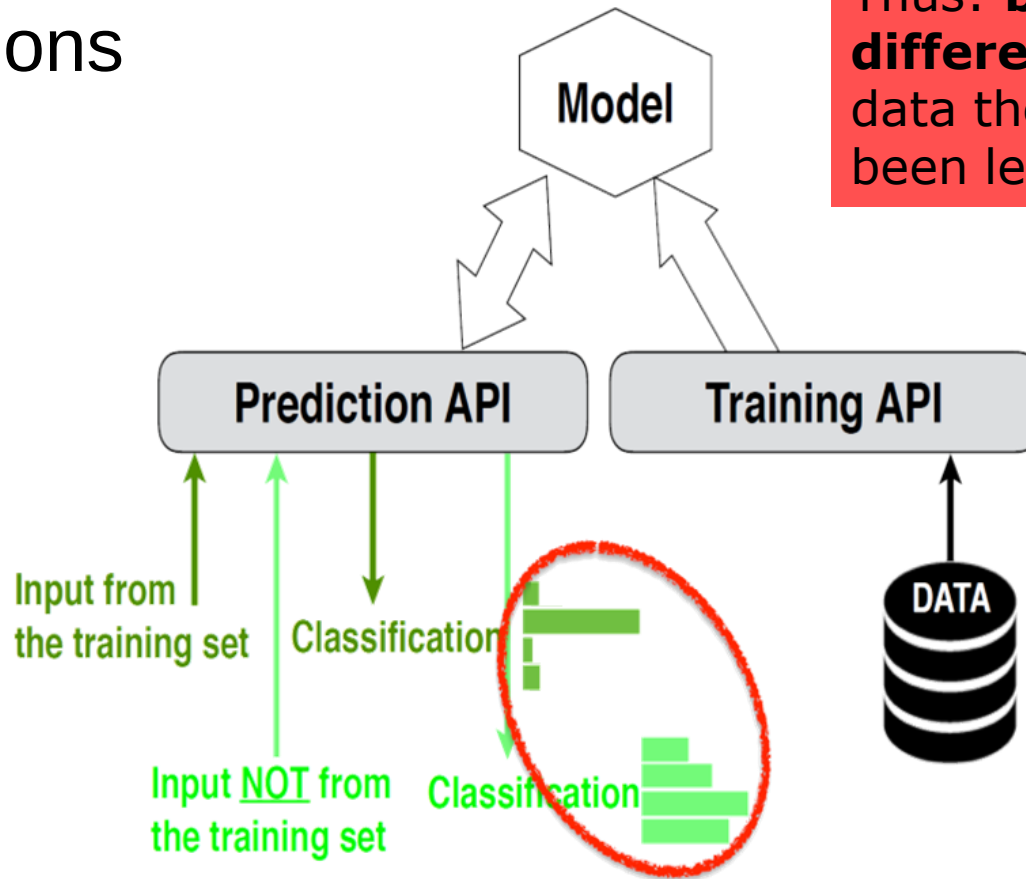
Membership Inference Attack

- Exploit Model's Predictions



Membership Inference Attack

- Exploit Model's Predictions

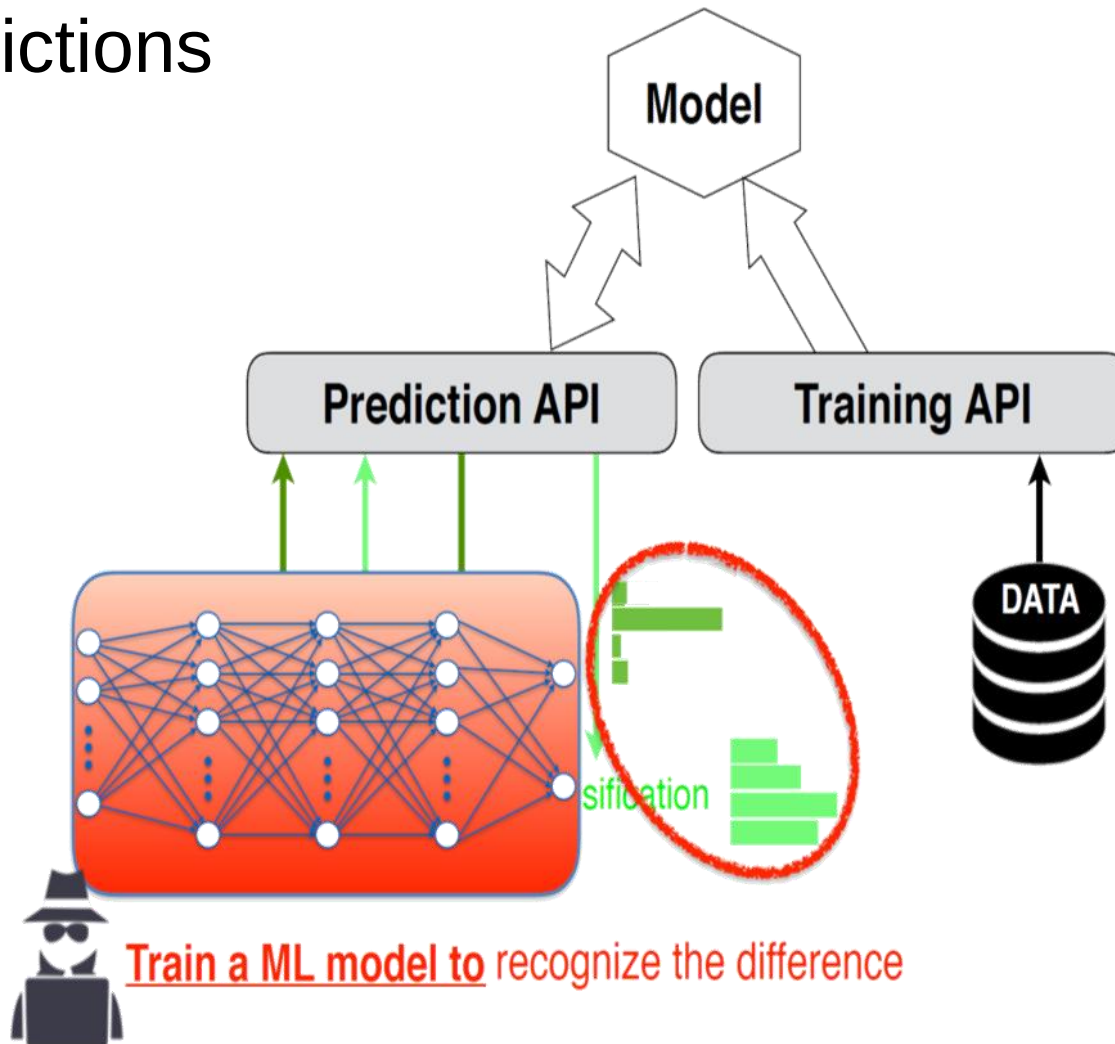


Thus: **behave differently** on data they have been learned on

Recognize the difference

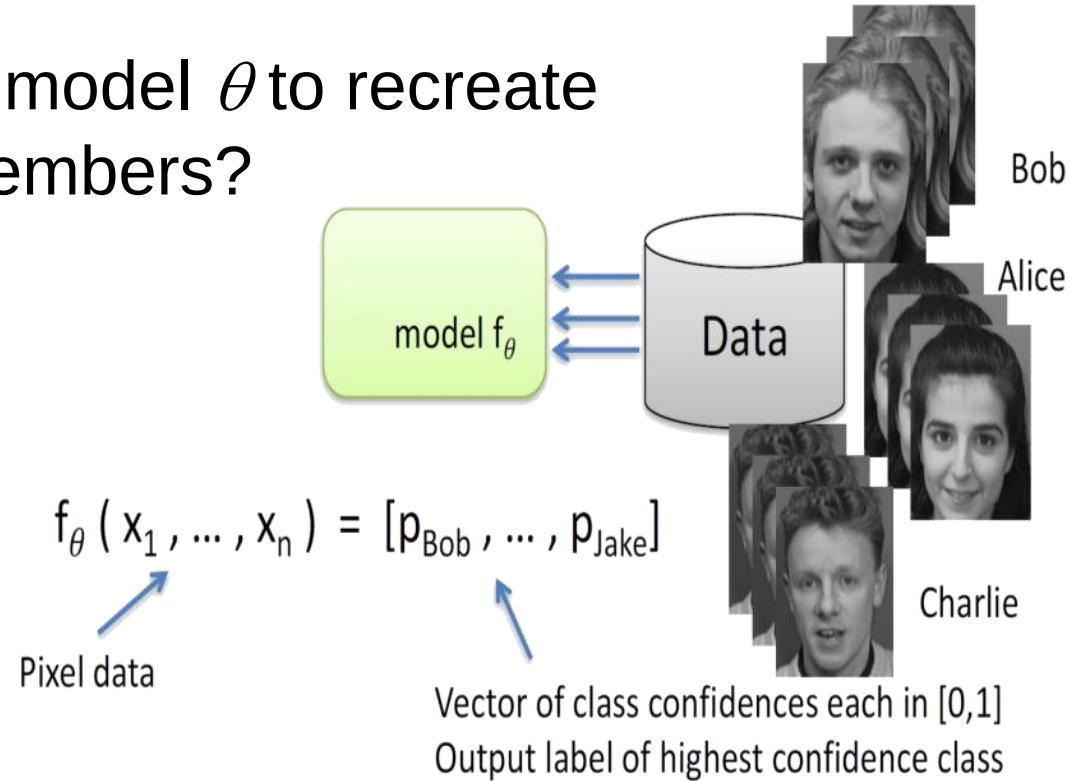
Membership Inference Attack

- Exploit Model's Predictions



Model Inversion: Face recognition

- Can Adversary uses model θ to recreate images of training members?



- Approach (slightly simplified)
 - Given $(\theta, y') = \text{"Bob"}$: find input x that is most likely to match "Bob"
 - Search for x that maximizes p_{bob} $\rightarrow$ brute force doesn't scale
 - Can search efficiently using (inverted) gradient descent
 - Can repeat for all class labels

- AT&T faces dataset (40 individuals, 400 images)
- Inversion for three classifiers
 - Multinomial LR, Multi-layer Perceptron, Denoising auto-encoder



Target



Softmax



MLP



DAE

- Mechanical Turk experiments: re-identify person up to 95% accuracy

Model Inversion: Face recognition

- A lot of novel research, using e.g. GANs to limit search space to more realistic examples

Real Samples



Attack Samples

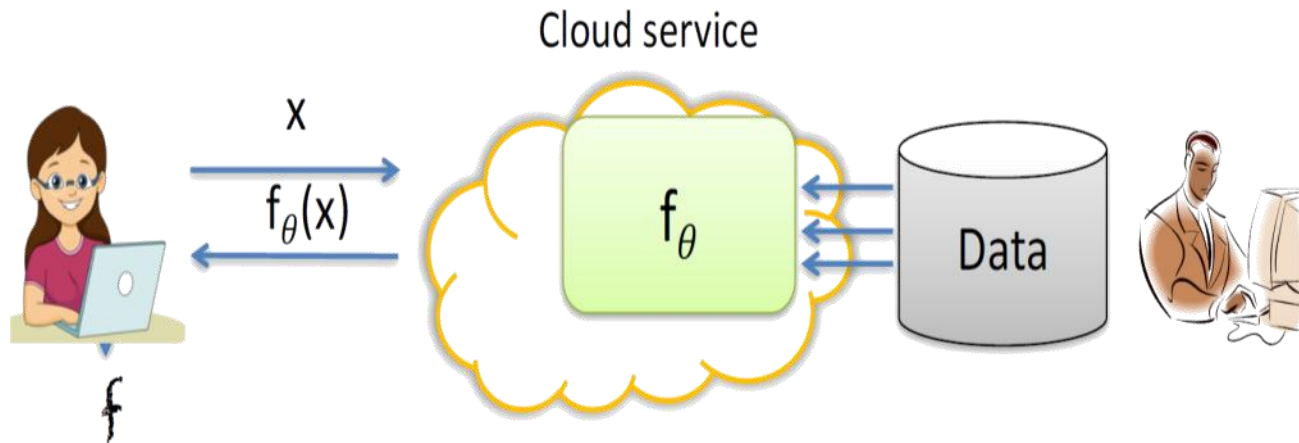


- *Scary, but (at the moment?) limited to specific settings*

Model Extraction/Stealing

- Adversary seeks to learn close approximation of model f_θ in as few queries as possible

Target: $f'(x) = f_\theta(x)$
on $\geq 99.9\%$ of inputs

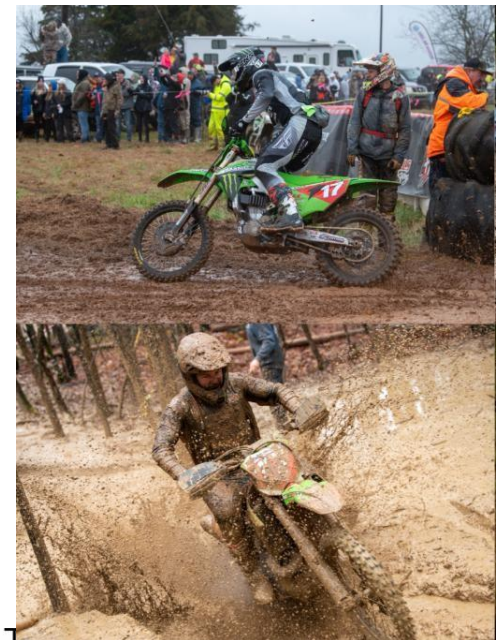


- Realistic attack? MLaaS! (e.g. Google, AWS, ..)*
- Efficient attacks could
 - Undermine pay-for-prediction pricing model – IP violation!
 - Enable evasion attacks
 - Facilitate privacy attacks

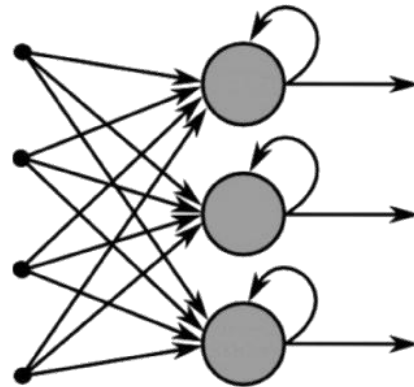
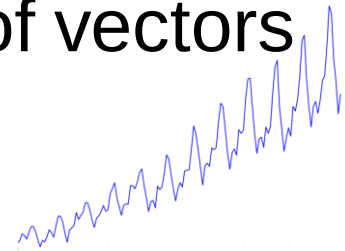
Tramèr et al. Stealing Machine Learning Models via Prediction APIs. USENIX Security 2016

- Short Recap
- Ensemble Learning
- Challenge: Robustness / Adversarial ML
- Challenge: Explainable AI
- Challenge: Privacy Preserving Machine Learning
- Recurrent Neural Networks

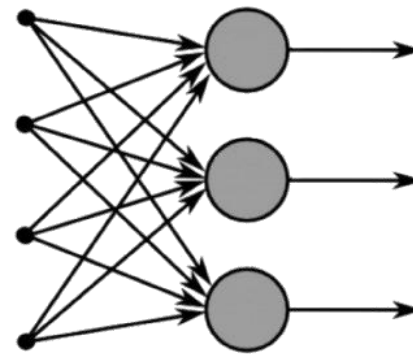
- Feed forward networks accept fixed-sized vector as input and produce fixed-sized vector as output
 - E.g. train a CNN for ImageNet
 - 224x224x3 input dimensionality
 - 1000 dimensional output: classes
- These are fixed, can't easily adapt to different sizes
 - If **some** images are larger / smaller:
 - Down-/upscale images
 - If **generally** larger / smaller images:
 - Train / fine-tune model with fitting input layer
 - Different output dimensionality?
 - I.e. different number of target classes
 - Train/fine-tune with new output layers (and potentially further FC layers)



- Feed forward networks: **fixed-sized** input & output
 - Fixed amount of computational steps
- Recurrent nets operate over sequences of vectors
 - Useful for sequential / time series data
 - Text, sound, video, ...



Recurrent Neural Network



Feed-Forward Neural Network

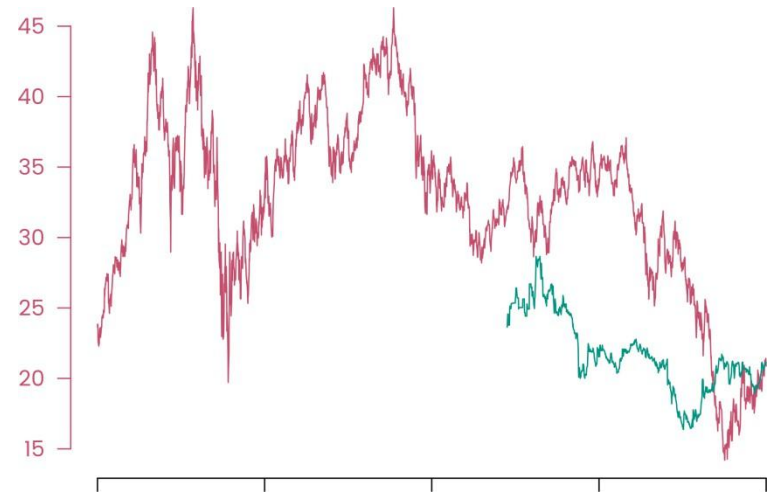
- How to feed stock-exchange data to MLP?
- Just consider the current day
 - Lose all context / history / memory of previous days – but they likely influence the prediction!
- Consider a fixed-length window, e.g. past 5 days
 - Very limited; difficult to choose window length
- Compute statistics over all previous days
 - Mean, variance, or higher-order moments,



- How to feed stock-exchange data to MLP?

Input is a sequence, variable length

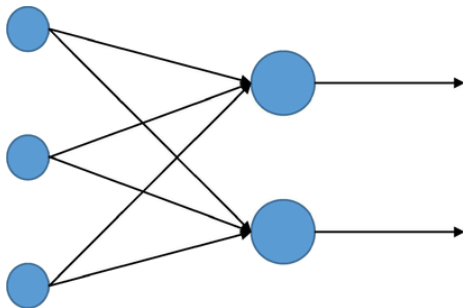
- E.g. companies started trading at different points in time



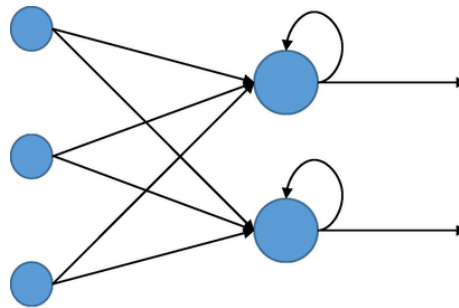
- Use models that can handle sequences per design – enter Recurrent Neural Networks!

- Convolutional Networks (Feed-Forward Networks)
 - Neurons organised in layers
 - Weights, bias & activation functions
 - Input: fixed-sized vector/tensor (e.g. an image of 224x224x3)
 - Output: fixed-sized vector/tensor (e.g. probabilities of classes)
- Recurrent Neural Networks:
 - Weights, bias & activation functions, **PLUS: feedback**
 - Thus: can operate over **sequences** of vectors/tensors
 - Input & output **size** is **variable** (“infinite”)
 - Have a **state** (“memory”)

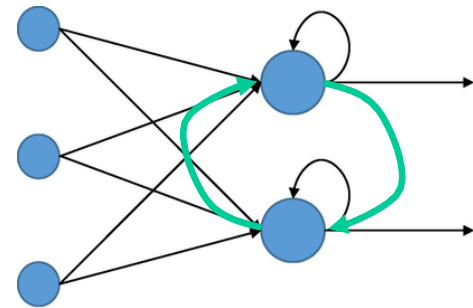
- Recurrent Neural Networks:
 - Weights, bias & activation functions
 - **Feedback via connections**
 - **Have a state (“memory”)**



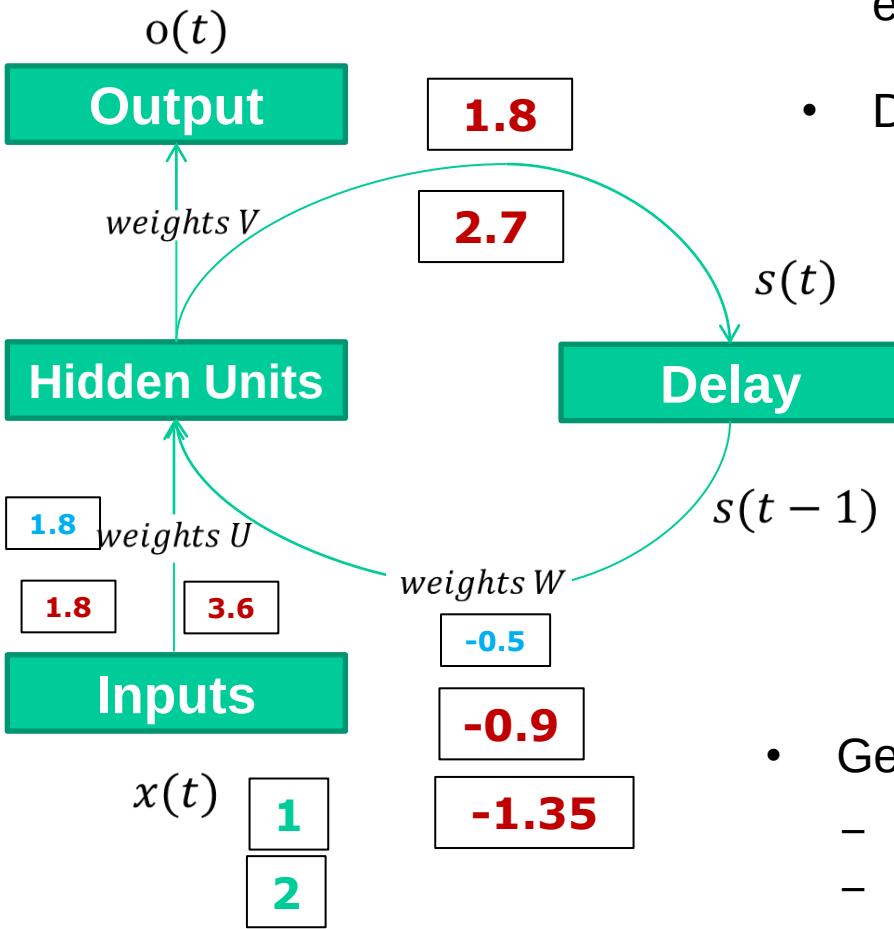
Feed forward
neural network



Simple
recurrent
neural
network

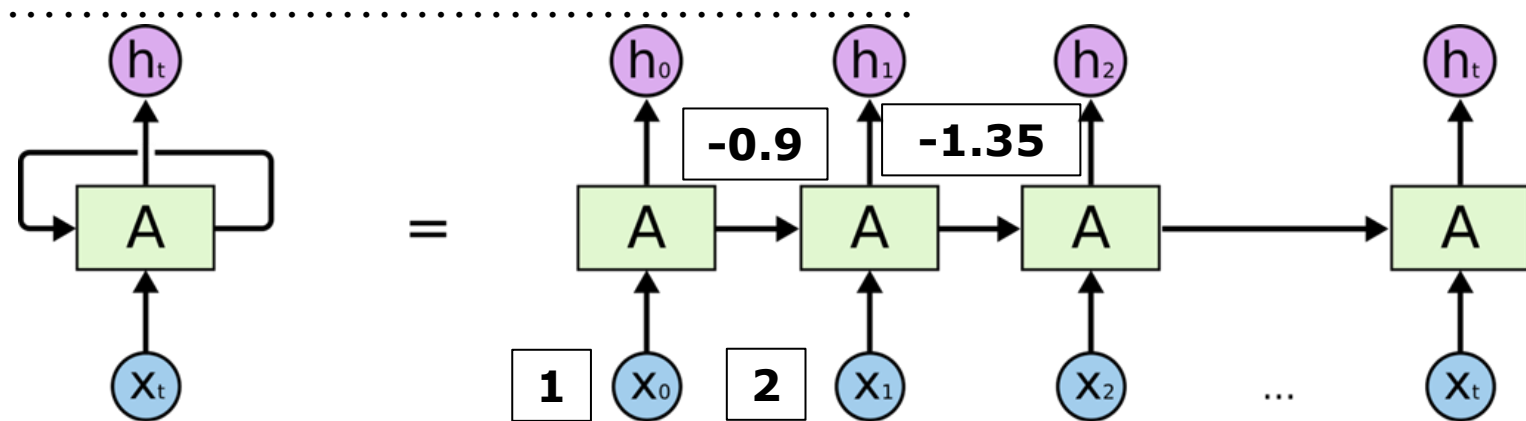


Fully connected
recurrent neural
network



- Recurrent, as they perform the same task for every element of the sequence
- Delay unit: holds **activation** until next step
 - Recurrent units: 2 sets of weights & inputs
 - **Weights:** U (input) & W (memory)
 - **Inputs** $x(t)$ (t-th element of the sequence) and $s(t-1)$ (memory)
 - Last layer: feed-forward layer
- Generally: only one set of W
 - Weights are reused! **Consequence?**
 - Number of parameters is independent of length of input
- Potentially also U and V are the same as W

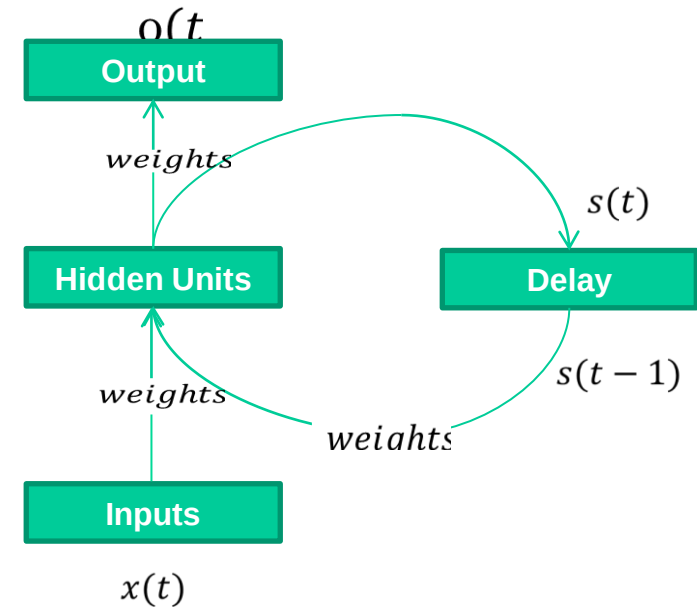
Recurrent Neural Networks (RNNs)



- RNNs process **sequences** of same-sized elements
- **Recurrent layers** process an input element together with their own output from the previous element
- Can be seen as a deep network with shared weights, unrolled in time
 - depth = length of sequence

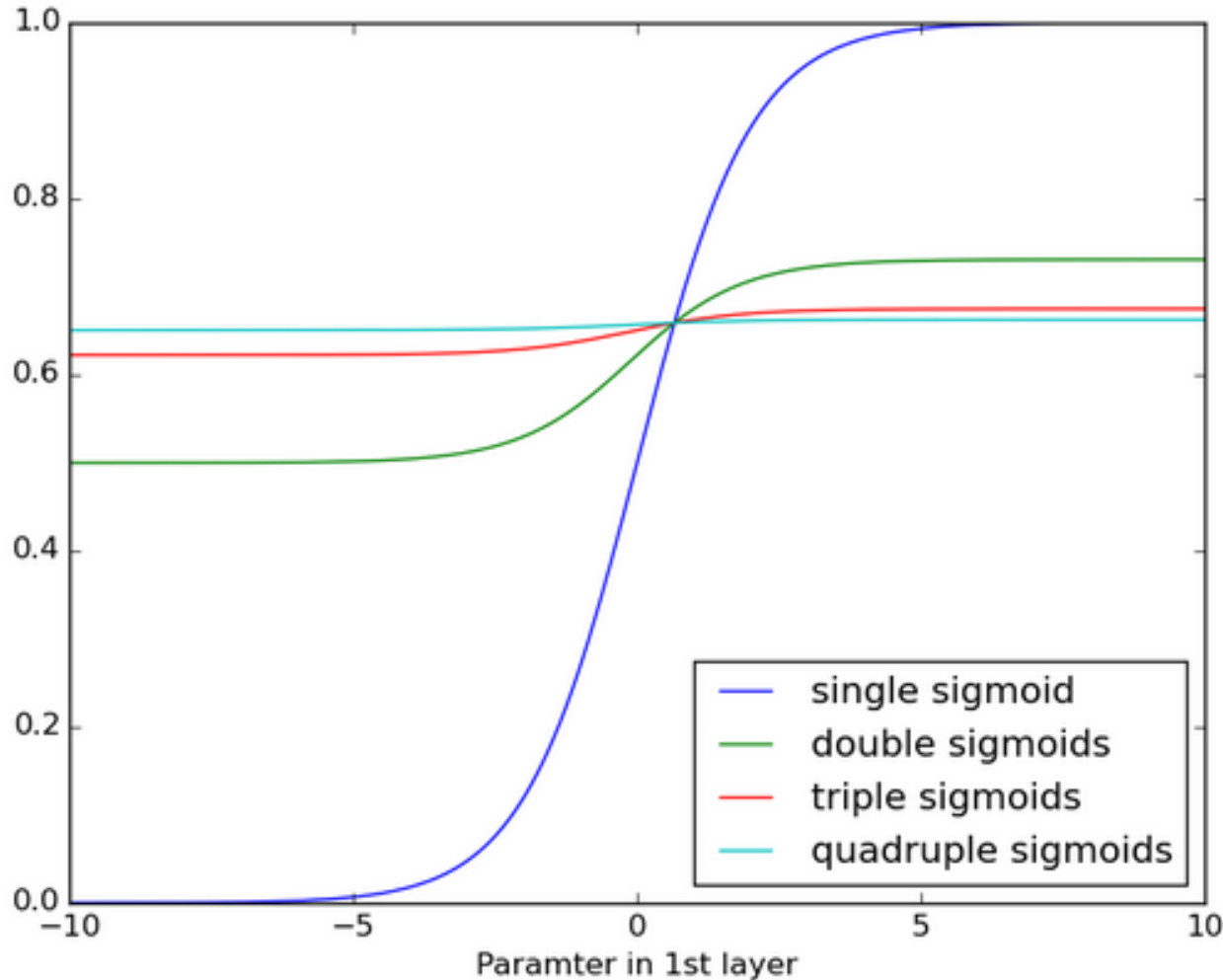
- Very similar to feed-forward networks (e.g. MLP)
- Forward pass: compute error/loss
 - On “unfolded” version of network
 - Intermediate steps & final output h_t
- Back-propagation through (in) time (BPTT)
 - Similar to MLP: derivatives, parameter updates
 - But on the unrolled version of the network
 - ➔ Cyclic graph transformed into an acyclic one
 - ➔ Can apply standard back-propagation

- Network input at time t :
 - $a_h(t) = Ux(t) + Ws(t-1)$
- Activation of input at time t :
 - $s(t) = f_h(a_h(t))$
- Output unit at time t :
 - $A_o(t) = V_s(t)$
- Output of network at time t :
 - $O(t) = f_o(a_o(t))$



- Long sequences: back-propagation consists of many steps (like a very deep network)
- **Consequences?**
 - Exploding / vanishing gradients
 - Each sequence element: multiplied by $w \rightarrow w^{|sequence|}$
 - E.g. 50 elements: $w > 1 \rightarrow ?$ $w < 1 ?$
 - Very large / close to 0
 - Exploding: “simple” solution of gradient clipping (e.g. max 1)
 - Exploding & Vanishing:
 - Specialised “cells” proposed
 - In architectures such as LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit)

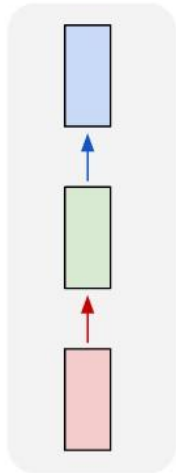
RNN: Vanishing Gradients



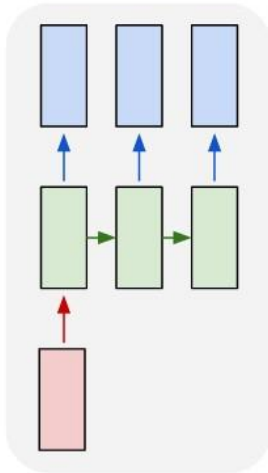
- Effect: does not learn long-term dependencies

- Sequential information (as **input** and/or **output**)

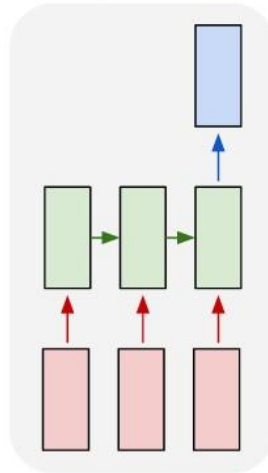
one to one



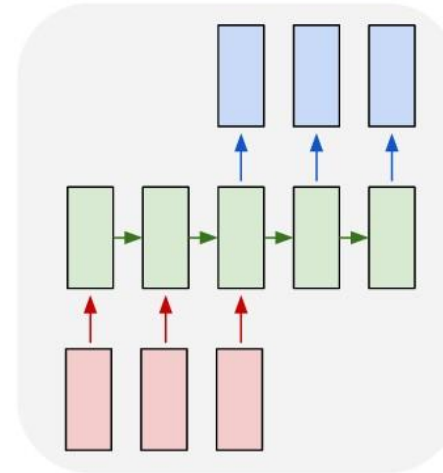
one to many



many to one



many to many



many to many

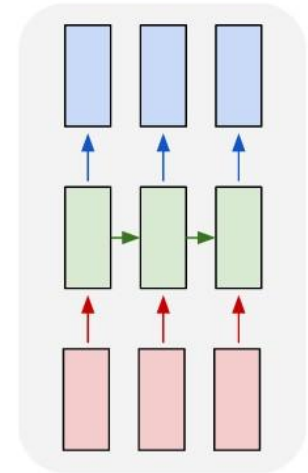


Image
classification
(one image $\rightarrow$ one label)
E.g. via CNN

Sentiment analysis
(many words $\rightarrow$ one label)

Synced sequence
(video classification, each frame)

Image captioning
(one image $\rightarrow$ many words)

Machine translation
(many words $\rightarrow$ many words)

- LSTM: Long Short-Term Memory
 - Don't keep the same feedback loop for early and recent inputs
 - E.g. not the same for the stock market price at the beginning of the sequence, and the one just yesterday
 - Instead, introduce different memories for long and short-term memories
 - More complicated unit than before

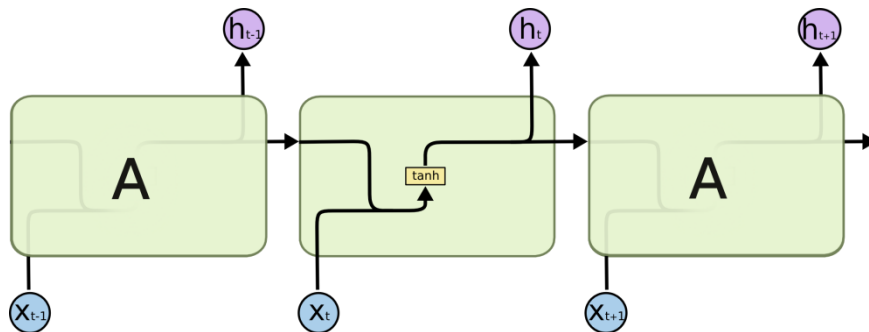


**SHORT-TERM
MEMORY**

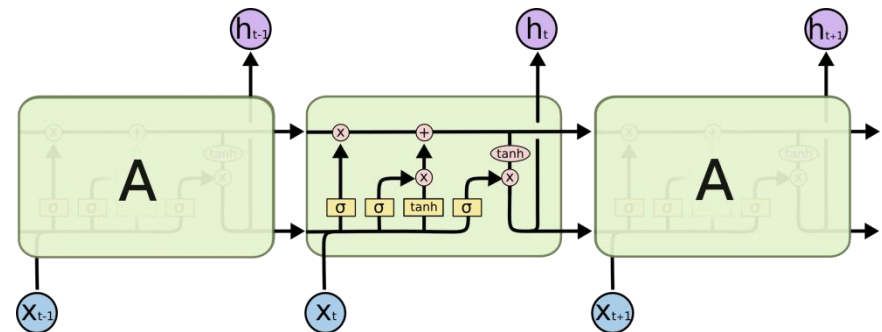


**LONG-TERM
MEMORY**

- LSTM: Long Short-Term Memory
 - Long-term memory (LTM): scalar, not multiplied by weight
 - Short-term memory (STM): multiplied by weight
 - Instead of neurons: memory blocks
 - ➔ Can store longer term memory
 - At each step: (i) keep old or (ii) update to new value



Standard RNN



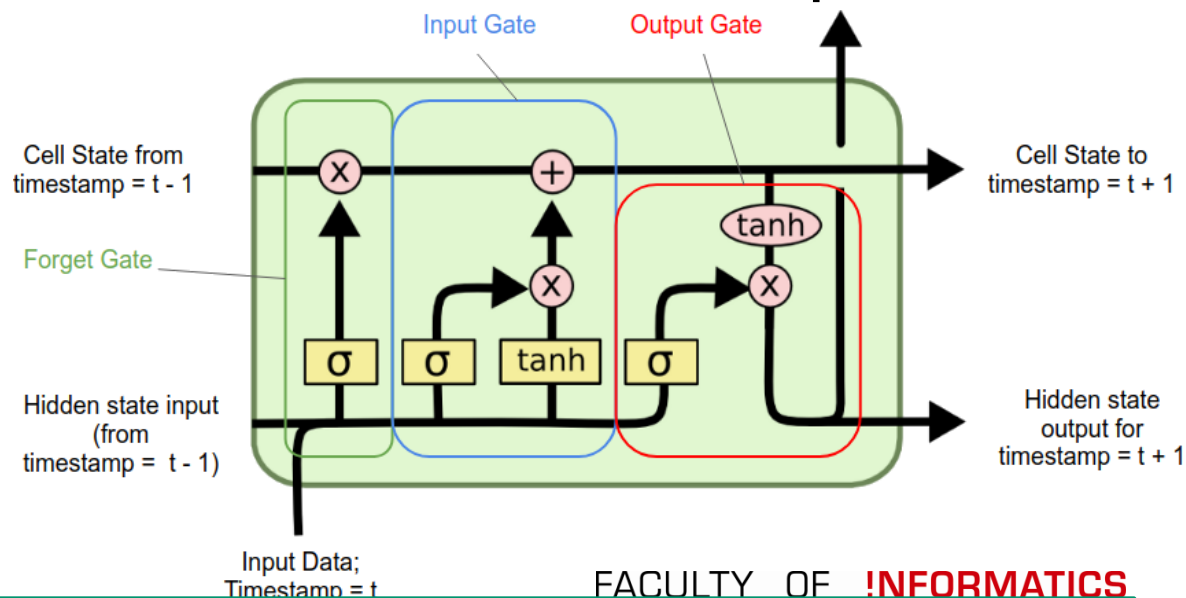
LSTM

contains 4 interacting layers

Details: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

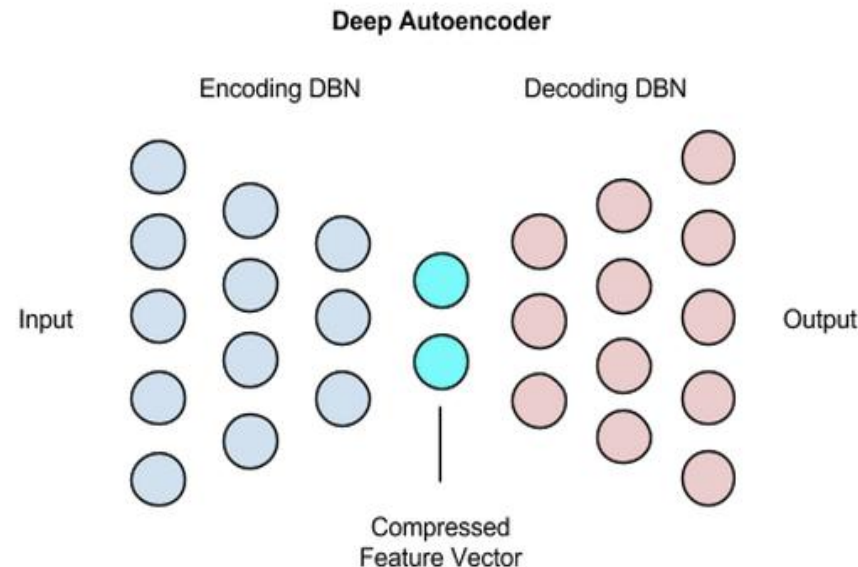
1. *Forget* gate computes how much of LTM to keep
 - *Forget*, though that gate computes how much to remember
2. Input gate (though it decides on how to update the memory...)
 1. Computes new potential LTM
 2. Multiplies by percentage of potential to keep (control)
3. Output gate decides on new STM & output

Different weights
for each gate



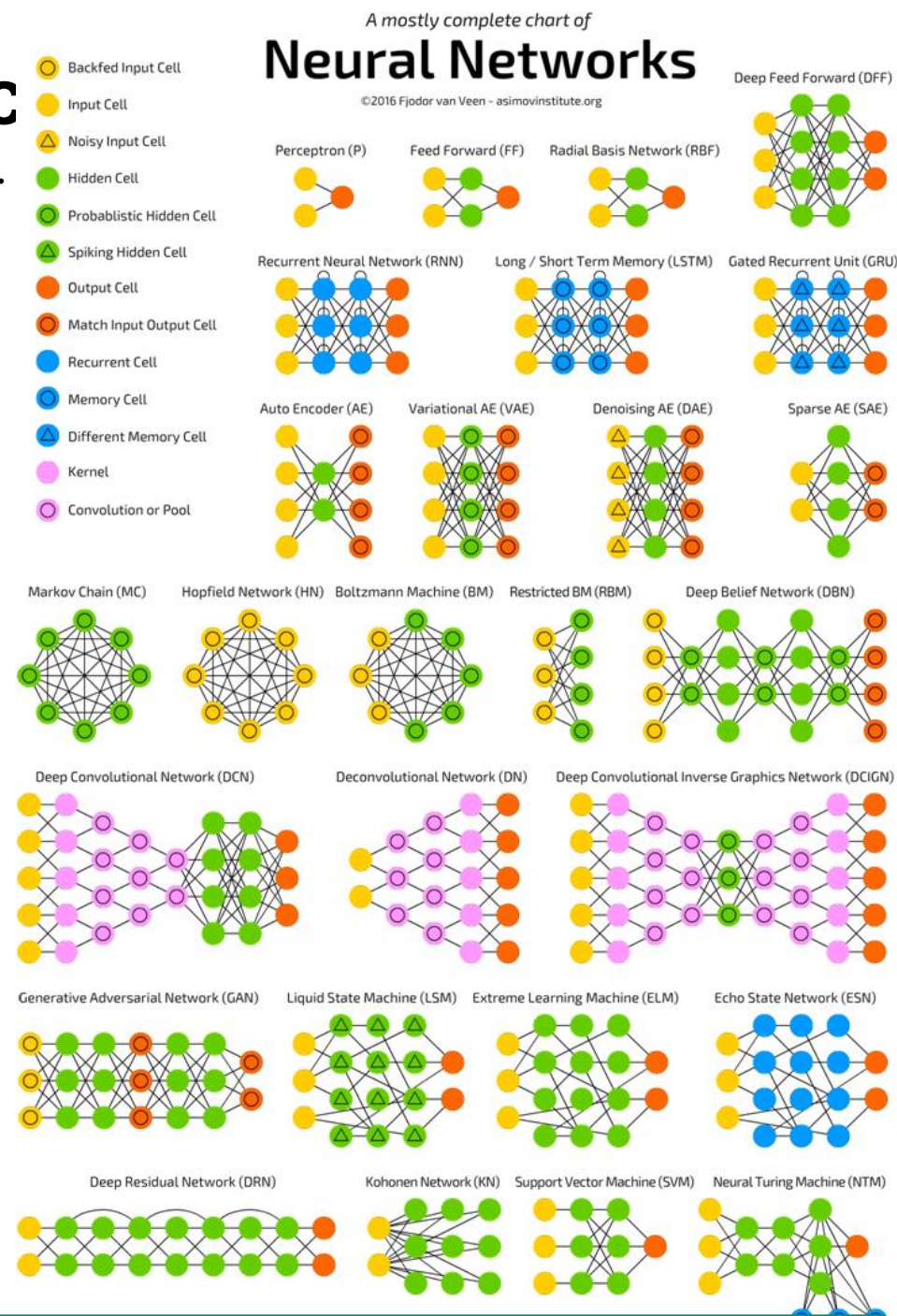
- LSTM: Long Short-Term Memory
- GRU: **Gated Recurrent Units**
 - Gating mechanism for RNNs (2014)
 - Similar performance as LSTM
 - But fewer parameters
- RNNs and CNNs can also be combined
 - e.g. an RNN processing sequences of CNN outputs

- Neural Network Type where input and output are nearly the same
- How is that useful? How is it different of just using the identity function?
- Purpose: typically dimensionality reduction / compression
- Learn a representation (encoding) for a data set

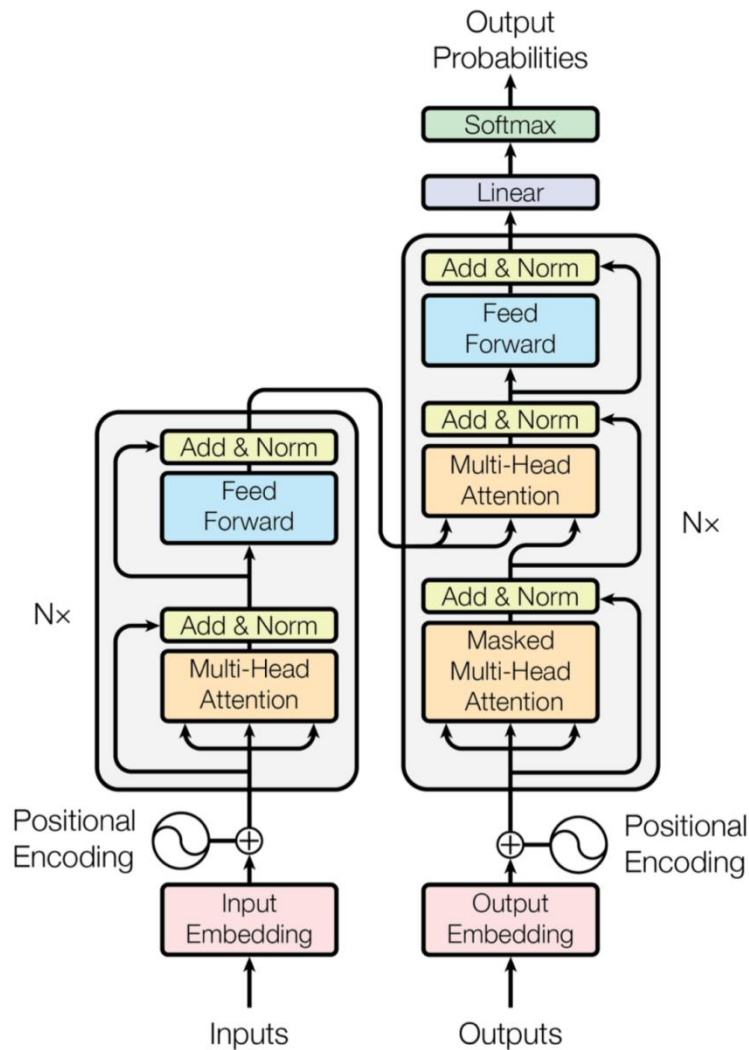


- Plenty of different architectures
 - For specific tasks
 - Supervised & unsupervised
 - Very active research field
 - Chart does e.g. not include Transformers (BERT; GPT, ...)

- <https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464>



- BERT & Transformers



.....

Questions ?